Hierarchical Active Inference using Successor Representations

Prashant Rangarajan[1], **Rajesh P. N. Rao**[1,*]

[1]Paul G. Allen Center for Computer Science and Engineering, University of Washington, Seattle, USA

[*]Corresponding author

Keywords: active inference, free energy principle, hierarchy, planning, successor representation, reinforcement learning

Abstract

Active inference, a neurally-inspired model for inferring actions based on the free energy principle (FEP), has been proposed as a unifying framework for understanding perception, action, and learning in the brain. Active inference has previously been used to model ecologically important tasks such as navigation and planning, but scaling it to solve complex large-scale problems in real-world environments has remained a challenge. Inspired by the existence of multi-scale hierarchical representations in the brain, we propose a model for planning of actions based on hierarchical active inference. The model combines a hierarchical internal model of the environment with successor rep-

resentations for efficient planning. We present results demonstrating (1) how lower-level successor representations can be used to learn higher-level abstract states, (2) how planning based on active inference at the lower-level can be used to bootstrap and learn higher-level abstract actions, and (3) how these learned higher-level abstract states and actions can facilitate efficient planning. We illustrate the performance of the model on several different planning problems including a variant of the well-known four rooms task and a key-based navigation task. Our results represent, to our knowledge, the first application of learned hierarchical state and action abstractions to active inference in FEP-based theories of brain function.

1 Introduction

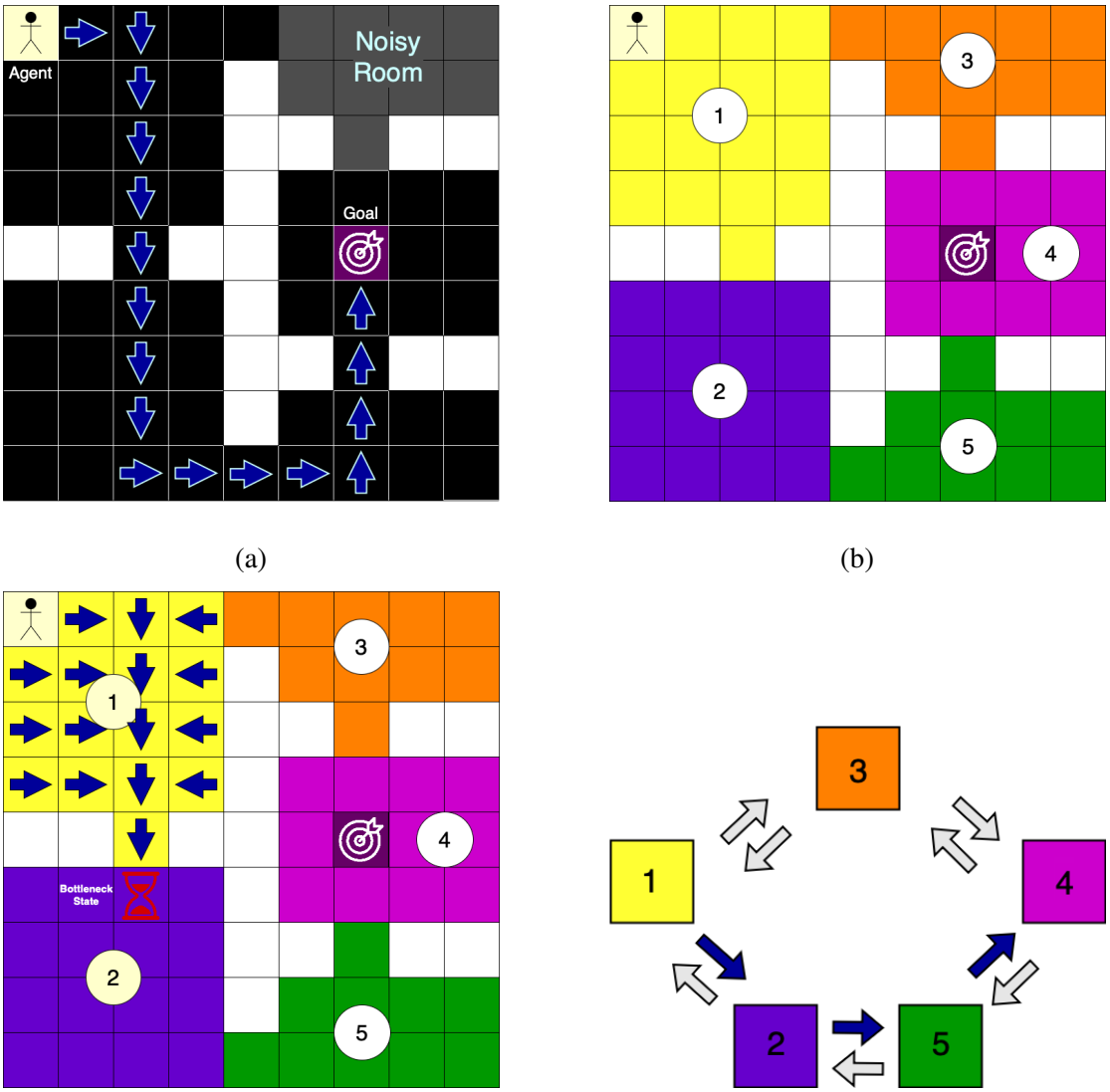
Active Inference is a biologically-inspired theory for explaining how agents make decisions in dynamic environments (Friston et al., 2015): an agent is modeled as being constantly engaged in a perception-action loop, with actions that can modify the state of the agent and the environment, causing new sensory observations to be received. In order to maintain a form of homeostasis of its internal states in relation to its environment, the agent is hypothesized to minimize its variational free energy, consistent with the free energy principle (FEP) (Friston, 2010). In this view, the agent can be thought of as an inference engine constantly minimizing the "surprise" of its sensory observations using approximate Bayesian inference to infer not only the hidden sensory state of the environment, but also actions that serve to minimize surprise, and hence, the free energy. This inference process includes a bias (Sharot, 2011) towards the agent's

preferences or prior beliefs.

The FEP, through active inference, can thus be regarded as extending brain theories such as predictive coding (Huang & Rao, 2011; Rao & Ballard, 1999) and other Bayesian brain models (Aitchison & Lengyel, 2017; Chater et al., 2020; Knill & Pouget, 2004; Rao et al., 2002) to action inference. The FEP treats action and perception as two different means of achieving the same objective: minimizing variational free energy to reduce uncertainty about the world.

Active inference algorithms use planning methods to choose actions that minimize the expected free energy of future observations with respect to the agent's preferred goal. In a discrete setting (Da Costa et al., 2020), the agent chooses among a set of actions to find the best sequence of actions that minimizes expected free energy, combining an intrinsic component (capturing epistemic value or expected information gain) and an extrinsic component (capturing instrumental value or expected utility, in terms of prior preferences or goals). Active inference thus offers a natural solution to the exploration-exploitation problem in reinforcement learning (Fountas et al., 2020; Friston et al., 2015). However, the elegance of active inference comes at a cost: it is hard to scale active inference to large-scale environments. Here we address this problem by proposing a hierarchical version of active inference based on successor representations (Dayan, 1993). We provide several examples of its applicability to hierarchical planning problems.

Figure 1 illustrates our proposed hierarchical active inference method based on successor representation using a simple "gridworld" environment. The central idea of our approach is to use a state and action hierarchy to reach a goal more efficiently. The



(a) (b)

(c) (d)

Figure 1: **Hierarchical Active Inference using Successor Representations.**

(continued next page)

Figure 1: (contd.)

(a) A 9×9 grid world with five rooms (separated by white walls with openings). The agent must navigate from the start state (top left) to the goal state (purple target symbol) to get a reward. In this example, one of the rooms has noisy observations. Active inference in this case would prefer an alternate (longer but less noisy) path to the goal. (b) By clustering of the successor matrix, the states of this gridworld can be grouped into 5 macro states (numbered) corresponding to the five rooms in the environment. (c) Example of a macro action learned by the model that corresponds to a policy for navigating from macro state 1 to macro state 2. The arrows represent the best action to take from each state in macro state 1 to get to a bottleneck state in macro state 2 (end point represented by the red hourglass). (d) Higher level planning. The arrows represent possible macro actions that the agent can take from one macro state to another. The blue colored arrows represent the optimal path chosen by applying active inference at the higher level to reach the room containing the goal location.

state-action hierarchy is learned by first clustering the successor matrix to obtain ‘macro states‘, and then learning ‘macro actions’ to navigate between these macro states. The macro actions allow the agent to plan at a higher level of abstraction, thereby enabling it to reach a goal more efficiently. The successor matrix is also used directly for planning at the lower level (instead of more traditional active inference-based planning), allowing the agent to significantly cut down the computational cost.

2 Related Work

Friston and colleagues have explored how active inference can be applied to deep hierarchical Bayesian models for goal-directed behavior (Friston et al., 2018; Pezzulo et al., 2018). Related work has focused on potential neural implementations of such mechanisms, for example, in hippocampal-prefrontal circuits (Proietti et al., 2023; Van de Maele et al., 2023). In these models, hierarchy emerges from an efficient factorization of the underlying Bayesian model. In contrast, our approach induces a hierarchy of states and actions from the successor representation.

Hierarchical planning has been extensively studied in the field of reinforcement learning (RL). In hierarchical reinforcement learning (HRL) (Dayan & Hinton, 1992; Pateria et al., 2021), macro actions called “options” (Sutton et al., 1999) have been proposed as temporally extended actions that enable efficient learning and planning in complex environments. In model-based RL, deep neural networks have been used to break down complex tasks efficiently for hierarchical planning (Hafner et al., 2022). Other researchers in RL have explored the use of state abstraction for solving Markov

Decision Processes (e.g., (Li et al., 2006)). Our model builds on these past approaches to leverage both state and action hierarchies for efficient planning.

Our approach uses the successor representation (Dayan, 1993), which has been proposed as a compromise between the computational inefficiencies of learning full-fledged models for model-based RL and the inflexibility of model-free RL. The successor representation has also been used for modeling human behavior (Momennejad et al., 2017). Correspondences have been identified between place cells in the hippocampus and the successor representation (Gershman, 2018; Momennejad, 2020; Stachenfeld et al., 2017; Stoewer et al., 2022) (see also (Whittington et al., 2020)). In the context of active inference, the concept of inductive inference (Friston et al., 2025a) bears some similarities with the successor representation: a known (or learned) series of transition matrices is collapsed over the action-dependent dimension of the transition tensor to provide a description of plausible transitions, which is used to implement a form of backwards induction. Other recent work has explored combining the successor representation with active inference (Millidge & Buckley, 2023) as a way to minimize computational cost, albeit without hierarchical states or actions.

3 Methods

3.1 Active Inference

Notation and Formulation

In this work, we assume that an agent does not have complete visibility of the state of the world. At each time step the agent uses the observations it gathers from sensors to refine

its internal estimate of the state of the world. It also executes actions at each time step to actively learn more about the state and reduce its uncertainty about the environment. Specifically, we model the problem using a Partially Observable Markov Decision Process (POMDP), where s_t, a_t, and o_t represent the hidden state, action and observation (or outcome), respectively, at time t for $1 \leq t \leq T$. Here T represents the planning horizon, that is, the number of future time steps the agent considers for planning. At each time t, the agent maintains a posterior probability distribution ("belief state") over the entire space of possible states as an estimate of its current state. We denote b_t as this belief state of the agent at time t. Although the underlying process is Markovian, the belief state accumulates information from all past observations to compute the posterior probability of state. Note that a Markov Decision Process (MDP), which forms the basis for traditional RL, can be regarded as a special case of the POMDP where the true state of the environment is observed at each time step (i.e., no noise in the observations). The Bayesian generative model of the agent is a joint distribution of states, observations, and actions over all time steps, represented by $p_\theta(o_{1:T}, s_{1:T}, a_{1:T-1})$ where θ represents the parameters.

Using Bayesian inference to determine the exact posterior probability distribution of states and actions under a full generative model as above may be intractable in many settings. In such cases, active inference employs variational inference to estimate an approximate posterior by minimizing the variational free energy. Specifically, a variational distribution $q_\phi(\cdot)$ (parameterized by ϕ) is used to approximate the posterior (note however that for the simple POMDP task we explore in this paper (Section 4.3), the posterior can be computed exactly and we therefore do not need ϕ).

The *surprisal* of an observation, given by $-\log p_\theta(o)$, quantifies how unlikely an observation is under a given generative model. Any agent that desires to remain in equilibrium (homeostasis) with its environment must minimize its surprisal over its set of observations. In accordance with the Free Energy Principle (Friston et al., 2006), the agent minimizes its (variational) free energy as a proxy in order to remain in homeostasis. The variational free energy (VFE) at time step t is defined as:

$$F_\pi(t) = \mathbb{E}_{q_\phi(s_t|\pi)}[\log q_\phi(s_t|\pi) - \log p_\theta(s_t, o_t|\pi)] \tag{1}$$

where π denotes a "policy," defined as a sequence of actions given by a plan (note that this definition of a policy differs from the definition of policy in reinforcement learning (Sutton & Barto, 2018) as a state-to-action mapping).

Expanding the second log term in Equation 1, VFE can also be written as:

$$\begin{aligned} F_\pi(t) &= -\log p_\theta(o_t) + \mathbb{E}_{q_\phi(s_t|\pi)}[\log q_\phi(s_t|\pi) - \log p_\theta(s_t|o_t, \pi)] \\ &= -\log p_\theta(o_t) + D_{KL}(q_\phi(s_t|\pi)||p_\theta(s_t|o_t, \pi)) \\ &\geq -\log p_\theta(o_t) \end{aligned}$$

where D_{KL} denotes the Kullback-Leibler divergence. As evident above, minimizing VFE is equivalent to minimizing an upper bound on the surprisal of the agent (this is similar to maximizing the evidence lower bound (ELBO) in variational autoencoders (Kingma & Welling, 2022)).

It is useful to consider an alternative way of expressing VFE, obtained by rearrang-

ing the terms in Equation 1 as follows:

$$
\begin{aligned}
F_\pi(t) &= \mathbb{E}_{q_\phi(s_t|\pi)}[\log q_\phi(s_t|\pi) - \log p_\theta(o_t, s_t|\pi)] && (2)\\
&= \mathbb{E}_{q_\phi(s_t|\pi)}[\log q_\phi(s_t|\pi) - \log(p_\theta(o_t|s_t, \pi)p_\theta(s_t|\pi))]\\
&= \mathbb{E}_{q_\phi(s_t|\pi)}[\log q_\phi(s_t|\pi) - \log p_\theta(o_t|s_t) - \log p_\theta(s_t|\pi)]\\
&= -\underbrace{\mathbb{E}_{q_\phi(s_t|\pi)}[\log p_\theta(o_t|s_t)]}_{accuracy} + \underbrace{D_{KL}(q_\phi(s_t|\pi)||p_\theta(s_t|\pi))}_{complexity} && (3)
\end{aligned}
$$

Thus, VFE can be viewed as quantifying the trade-off between accuracy and model complexity. Specifically, the first term in Equation 3, accuracy, measures how well the agent's model of the world predicts the sensory observations. The second term, representing the complexity of the model, quantifies how close the posterior state distribution remains to the prior. Thus, minimizing VFE can be understood as maximizing the accuracy of one's predictions of observed outcomes while at the same time trying not to increase the complexity of the model.

The VFE as defined above depends only on the current state and observation. In planning, the agent selects an optimal sequence of future actions which requires us to estimate the free energy for future actions and observations. This entails a formulation of free energy known as expected free energy (EFE), which incorporates future observations and actions, for determining the optimal policy (Millidge, 2020).

The EFE at time τ ($t \leq \tau \leq$T) is defined as:

$$
G_\pi(\tau) = \mathbb{E}_{q_\phi(o_\tau, s_\tau|\pi)} \log[q_\phi(s_\tau|\pi) - \log p_\theta(o_\tau, s_\tau|\pi)] \tag{4}
$$

The structure of EFE is very similar to VFE (compare the above equation to Equation 2). However, unlike VFE, EFE is used for planning and the observations occur at future time steps. The observation o_τ at a future time step τ is therefore treated as a random

variable, and the expectation operation in Equation 4 includes future observations in addition to future states. Active inference picks actions that minimize the EFE in the future. We can simplify the right side of Equation 4 further and get:

$$
\begin{aligned}
G_\pi(\tau) &= \mathbb{E}_{q_\phi(o_\tau, s_\tau|\pi)}[\log q_\phi(s_\tau|\pi) - \log p_\theta(s_\tau|o_\tau, \pi)] - \mathbb{E}_{q_\phi(o_\tau|\pi)} \log p_\theta(o_\tau|\pi)] \\
G_\pi(\tau) &\approx -\underbrace{\mathbb{E}_{q_\phi(o_\tau, s_\tau|\pi)}[\log q_\phi(s_\tau|o_\tau, \pi) - \log q_\phi(s_\tau|\pi)]}_{expected\ information\ gain} - \underbrace{\mathbb{E}_{q_\phi(o_\tau|\pi)}[\log \tilde{p}_\theta(o_\tau)]}_{utility} \quad (5)
\end{aligned}
$$

The first term in Equation 5 is the (negative) epistemic value, which is the expected information gain or reduction in uncertainty when conditioned on a future observation. This represents the information seeking (or intrinsic) motivation of the agent. The second term in Equation 5 is the pragmatic value, which is the expected utility or score of an observation.

As discussed earlier, active inference selects actions that take the agent towards observations with high priors. In Equation 5 above, we use $\tilde{p}$ to represent the agent's prior probability distribution over sensory observations (instead of a probability distribution conditioned on policy π): this allows the EFE to capture the agent's prior biases for particular observations as being "desirable" (e.g., locations containing food or goal locations in navigation). Thus, the second term in the EFE equation above represents the reward seeking (or extrinsic) motivation of the agent to seek outcomes that match its prior preferences. Clearly the agent would like to minimize its uncertainty about the world through information gathering (first term in Equation 5), but it would also like to maximize its reward (second term in Equation 5). Thus, the EFE as defined in Equation 5 naturally captures the tradeoff between exploration and exploitation, a topic of considerable interest in reinforcement learning (Sutton & Barto, 2018).

A more convenient formulation of EFE for the purpose of computation is as follows:

$$G_\pi(\tau) = \underbrace{D_{KL}[q_\phi(o_\tau|\pi)||\tilde{p}_\theta(o_\tau)]}_{risk} + \underbrace{\mathbb{E}_{q_\phi(s_\tau|\pi)}[H(p_\theta(o_\tau|s_\tau))]}_{ambiguity} \tag{6}$$

where H denotes entropy. In the equation above, the first term represents the risk or expected cost/complexity, i.e., the KL-divergence between the predicted outcomes dictated by a policy π and the preferred outcomes dictated by the agent's prior $\tilde{p}_\theta(o_\tau)$ capturing its preferences or goals. The lower this value, the higher the chance of achieving a rewarding outcome based on the policy π. The second term represents the ambiguity or the expected entropy of the likelihood distribution. The lower this value, the more precise are the agent's predictions about its observations given its beliefs about the true state of the world. Policies that direct agents to states that provide precise observations about the world give a better picture about the true underlying state, and thus are desirable.

The path integral (or summation) of the EFE over all future time steps, given policy π, starting from the current time step t, and going up to the horizon T is given by:

$$G_\pi = \sum_{\tau=t}^{T} G_\pi(\tau) \tag{7}$$

By minimizing G_π over the space of all possible policies, the agent can identify the optimal sequence of actions to execute. Alternatively, rather than using a greedy approach where the agent selects the policy or action sequence that minimizes G_π, the agent can estimate the probability distribution over the policy space and sample a policy from it instead. Let the distribution over the policies be Q. Then, we can use the EFE values to compute the probability of a policy π as: $Q(\pi) = \sigma(-G_\pi)$ where σ is the softmax function $\sigma(x) = \frac{exp(x)}{\sum_x exp(x)}$.

Note that there are exponentially many such policies (as a function of the planning horizon T), making exact minimization computationally challenging for long horizons. One way to address this issue is to evaluate G_π over a shorter time horizon. Based on this evaluation, the agent can sample an action according to the marginal distribution $Q(a) = \sum_{\pi:a_t=a} Q(\pi)$, representing the probability of the action a being executed at the current time step t and the summation is over all policies (action sequences) beginning with a. After executing the sampled action, the agent can recompute G_π once again and take the next action as described above, and repeat this process until the goal state is reached (or the episode times out).

Bayesian Model for a Discrete POMDP

Consider the case of a POMDP which has the joint probability distribution $p_\theta(o_{1:T}, s_{1:T}, a_{1:T-1})$ with discrete states, actions and observations. Let us consider a simple case in which the set of possible states is $\mathcal{S} = \{s^{(1)}, s^{(2)}, \ldots, s^{(n_s)}\}$, the set of observations is $\mathcal{O} = \{o^{(1)}, o^{(2)}, \ldots, o^{(n_o)}\}$, and the set of actions is $\mathcal{A} = \{a^{(1)}, a^{(2)}, \ldots, a^{(n_a)}\}$ where n_s, n_o and n_a are the number of possible states, observations and actions respectively. We assume that the state and observation of an agent at time t are represented by one-hot vectors, where $s_t \in \mathbb{R}^{n_s \times 1}$ and $o_t \in \mathbb{R}^{n_o \times 1}$.

Due to underlying process being Markovian, we can factorize the joint distribution as follows:

$$p_\theta(o_{1:T}, s_{1:T}, a_{1:T-1}) = p_\theta(s_1) p_\theta(a_{1:T-1}) \left(\prod_{t=1}^{T} p_\theta(o_t|s_t) \right) \left(\prod_{t=2}^{T} p_\theta(s_t|s_{t-1}, a_{t-1}) \right) \quad (8)$$

This Bayesian model is visualized in Figure 2.

Parameters of a Discrete POMDP and Parameter Learning

The POMDP in Equation 8 is defined by the set of parameters θ which includes the following components (Figure 2) which can be set a priori or learned:

- The observation model or likelihood $p_\theta(o_t|s_t)$, which captures the underlying relationship between the observations (sensory measurements) and hidden states of the agent. It can be represented by a matrix $A \in \mathbb{R}^{n_o \times n_s}$, where each column represents the probabilities of different possible observations given a particular state. We assume that this probability distribution remains identical for all time steps and across all policies. In the case where there are multiple modalities, a separate A matrix can be maintained for each modality. If the A matrix is an identity matrix, then the model simplifies to an MDP, where the observation space and state space are identical. This is the case of our Tasks 1 and 2 below (Sections 4.1 and 4.2). For the POMDP Gridworld in Task 3 (Section 4.3) in which the observations are noisy versions of the true state, we assume this noise and hence A is known.

- The state-action transition probability distribution $p_\theta(s_t|s_{t-1}, a_{t-1})$, which encodes the Markovian dynamics of the model. It can be represented by $B \in \mathbb{R}^{n_s \times n_s \times n_a}$, a tensor which contains a state transition matrix for each action. We can also denote B_a as the $n_s \times n_s$ state transition matrix corresponding to action a. For the results reported in Section 4, we use a simple count-based learning procedure for learning B: for each occurrence of the triple $(s_t = s_i, s_{t-1} = s_j, a_{t-1} = a_k)$, we increment $B(i, j, k)$ by 1 (all entries of B are initialized to zero before the triples are obtained from interacting with the environment). Note that the learning of B could be formalized using Dirichlet distributions (e.g., (FitzGerald et al., 2015)), but for simplicity, we use the count-based procedure above for our

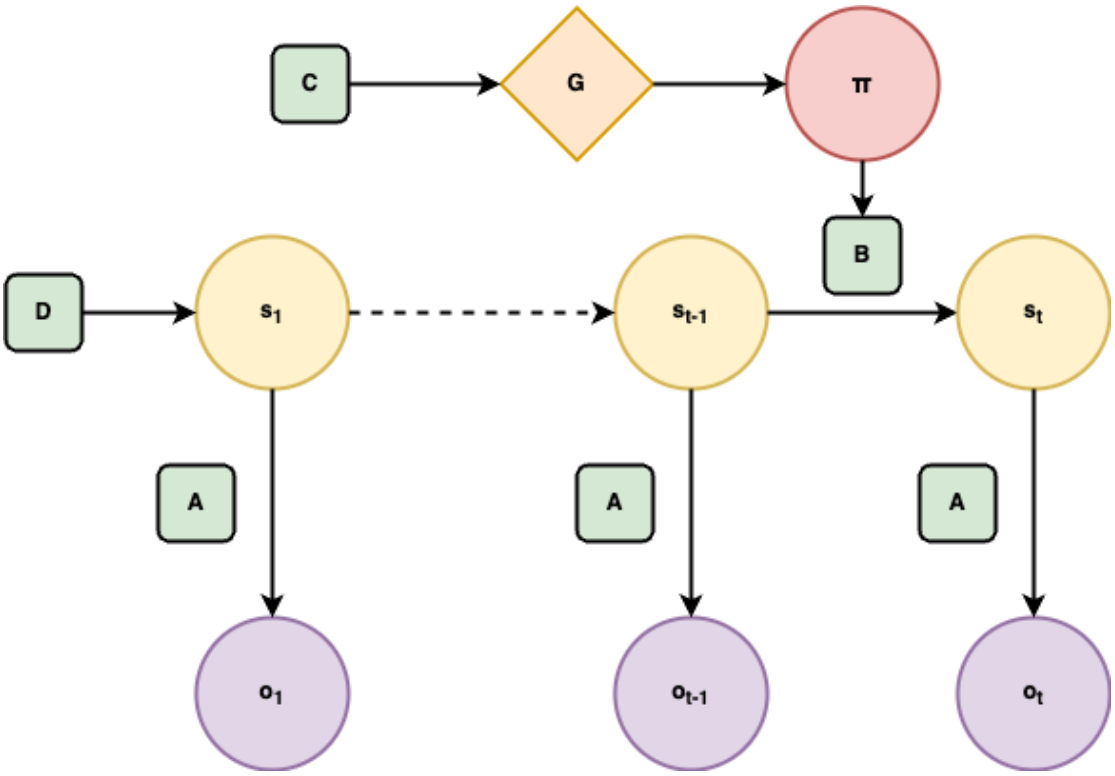


Figure 2: **Generative model for a Discrete POMDP.** The notation used here follows prior active inference literature (e.g., (Smith et al., 2022)). The EFE G depends on the prior C and generates the probabilities of different policies π. The policy π being a sequence of actions in turn affects the state transition matrix B. The observations are generated from the state using the observation model determined by the matrix A.

demonstrations in Section 4.

- Prior over observations, $\tilde{p}_\theta(o_t)$, represented by $C \in \mathbb{R}^{n_o \times 1}$. The agent seeks observations that have higher prior probability, i.e., are considered more rewarding. We assume this is known to the agent and therefore C is assumed to be known for the experiments in Section 4.

- Prior over initial state $p_\theta(s_1)$, which can be denoted by the vector $D \in \mathbb{R}^{n_s \times 1}$.

Since the state is encoded as a 1-hot vector, the actual prior probability for a particular initial state s_1 is given by $p_\theta(s_1) = s_1^\top D$. D is often assumed to be a uniform distribution over $\mathcal{S}$ if the agent has no other prior information. We make this assumption for the results in Section 4.

- Prior over policies $p_\theta(\pi)$ or $p_\theta(a_{1:T-1})$. This is represented by a vector E over the space of policies with the elements of the vector corresponding to the prior probability for each policy. Based on past actions or habits of the agent, the prior probability could be higher for certain policies. However, for the present work (and the results in Section 4), we assume a uniform prior over policies, leading to a constant factor for this term in the factorization in Equation 8.

Using the vector and matrix notation above, the factorization of the joint distribution in Equation 8 can be rewritten as:

$$p_\theta(o_{1:T}, s_{1:T}, a_{1:T-1}) \propto (s_1^\top D) \left(\prod_{t=1}^{T} o_t^\top A s_t \right) \left(\prod_{t=2}^{T} s_t^\top B_{a_{t-1}} s_{t-1} \right)$$

To infer the agent's hidden state in a POMDP environment, the agent can estimate a belief state $b_t \in \mathbb{R}^{n_s \times 1}$, which represents the posterior distribution over its possible states at time t:

$$b_t = \sigma \left(\ln A^\top o_t + \ln B_{a_{t-1}} b_{t-1} \right) \tag{9}$$

where the functions σ (the softmax function) and $\ln$ are applied to each element of their vector arguments individually. Note that in this case of a discrete POMDP where the exact posterior can be computed as above, we do not need an approximating distribution q or separate parameters ϕ.

For planning using active inference, the EFE in Equations 6 and 7 can be rewritten in vector-matrix notation as:

$$
\begin{aligned}
G_\pi(\tau) &= \underbrace{D_{KL}[q_\phi(o_\tau|\pi)||\tilde{p}_\theta(o_\tau)]}_{risk} + \underbrace{\mathbb{E}_{q_\phi(s_\tau|\pi)}[H(p_\theta(o_\tau|s_\tau))]}_{ambiguity} \qquad (10) \\
&= \left((As_\tau)^\top (\ln As_\tau - \ln C_\tau)\right) - \operatorname{diag}\left(A^\top \ln A\right)^\top s_\tau \\
G_\pi &= \sum_\tau \left((As_\tau)^\top (\ln As_\tau - \ln C_\tau)\right) - \operatorname{diag}\left(A^\top \ln A\right)^\top s_\tau
\end{aligned}
$$

As before, the agent selects a policy according to the probability distribution $Q(\pi) = \sigma(-G_\pi)$ where σ is the softmax function.

3.2 Successor Representation

Active inference requires a search over a sequence of future actions (for computing EFE as above) and as a result, does not scale well as the time horizon T increases. For problems that involve choosing actions to reach a desired goal state, the scaling problem for active inference can be ameliorated if we have an estimate of how close a particular state is to the goal state, in terms of reachability as defined by the environment's transition function. This notion of reachability can be quantified using the successor representation (Dayan, 1993), defined as a matrix M whose entries $M(s, s')$ represent the expected future occupancy of a state s' starting from the state s.

The successor representation was originally proposed in reinforcement learning as a way of quickly estimating the value function, which represents the expected cumulative future reward for any given state. Reinforcement learning algorithms rely on the value function for computing the policy of the agent: the best action to take in any state is

the one that will result in a state with the highest value. The problem arises when the goal or reward structure of the environment changes. In such a case, the agent needs to relearn the value function from scratch.

A more efficient approach is to factor the dynamics of the environment from the goals, and estimate the value function on the fly when given a new goal or task. Specifically, the value function can be decomposed into a product of the successor representation (the dynamics) and the reward function (goals). Thus, even if the reward structure changes, one can use a previously learned successor representation to quickly estimate the new value function and consequently the new policy.

Let s_t and r_t represent the state and reward obtained by the agent at time t. Also let γ be a discount factor. Let ρ be a state-to-action mapping (or "policy" as defined in the reinforcement learning sense (Sutton & Barto, 2018)). Note that ρ contrasts with the sequence-of-actions policy π used above for active inference: while reinforcement learning is based on the Bellman optimality principle and aims to optimize a function of states (the optimal policy) for maximizing expected future reward, active inference aims to optimise a functional of beliefs about states to evaluate a sequence of actions. Because beliefs about states can change during inference and learning, there is no unique state-to-action policy in active inference. The equivalent (Bellman) recursion in belief space entails a recursive estimation of EFE (Friston et al., 2021) that we elude using the successor representation (Dayan, 1993).

Given a state-to-action policy ρ, the value $v_\rho(s)$ of a state s is defined as:

$$v_\rho(s) = \mathbb{E}_\rho \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right] \tag{11}$$

We assume the reward function $r(\cdot)$ only depends on the current state s. Also, let $p(s'|s,a)$ be the transition probability from state s to s' using action a. Then, the value function can be defined recursively with the following "Bellman equation" (Sutton & Barto, 2018):

$$v_\rho(s) = r(s) + \sum_a \rho(a|s) \sum_{s'} \gamma p(s'|s,a) v_\rho(s') \tag{12}$$

The successor representation, which represents the expected cumulative discounted future occupancy of state s' from state s, is defined as:

$$M_\rho(s,s') = \mathbb{E}_\rho \left[\sum_{k=0}^{\infty} \gamma^k \delta(s_{t+k}, s') \mid s_t = s \right] \tag{13}$$

where δ is the Kronecker delta function equaling 1 if the two arguments are equal and zero otherwise.

Consider the case of a discrete MDP. Let $\mathbf{v} \in \mathbb{R}^{n_s}$ be a vectorized form of the value function with entries $v_\rho(s)$. Let $T \in \mathbb{R}^{n_s \times n_s}$ be the one-step state transition matrix for policy ρ whose entries $T_\rho(s,s')$ represent the probability $p(s'|s) = \sum_a \rho(a|s) p(s'|s,a)$. Let $\mathbf{r} \in \mathbb{R}^{n_s}$ be the reward function vector with entries $r(s)$. The Bellman equation (Equation 12) can be written in vector form as:

$$\begin{aligned} \mathbf{v} &= \mathbf{r} + \gamma T \mathbf{v} \\ &= \mathbf{r} + \gamma T[\mathbf{r} + \gamma T[\mathbf{r} + \gamma T[\cdots]]] \\ &= [I + \gamma T + \gamma^2 T^2 + \cdots]\mathbf{r} \end{aligned} \tag{14}$$

where I is the identity matrix. The successor matrix $M \in \mathbb{R}^{n_s \times n_s}$, whose entries are

$M_\rho(s, s')$ (see Equation 13), can be written as:

$$M = I + \gamma T + \gamma^2 T^2 + \cdots = \sum_{k=0}^{\infty} \gamma^k T^k = (I - \gamma T)^{-1} \tag{15}$$

Substituting M for the summation in Equation 14, we have:

$$\mathbf{v} = M\mathbf{r} \tag{16}$$

This equation can be written in non-vector form as:

$$v_\rho(s) = \sum_{s'} M_\rho(s, s') r(s') \tag{17}$$

Suppose the discrete states are numbered $1, \ldots, N$ and we are given a trajectory of states obtained by executing a policy ρ, for example, a uniformly random exploration policy. If the state at time step t in the trajectory is $s_t = i$, then the successor matrix can be learned using the temporal difference learning rule (Dayan, 1993):

$$M_\rho(i, j) = M_\rho(i, j) + \alpha(\delta(s_{t+1}, j) + \gamma M_\rho(s_{t+1}, j) - M_\rho(i, j)) \tag{18}$$

where α is the learning rate. Alternatively in vectorized form, for a state transition from s to s', each row s of M can be updated as follows:

$$M(s, :) = M(s, :) + \alpha(\mathbf{1}_{s'} + \gamma M(s', :) - M(s, :)) \tag{19}$$

where $\mathbf{1}_{s'}$ is a one hot vector with value 1 at the position corresponding to state s'.

If the state transition probability matrix T is known, either through learning by interacting with the environment or by prior knowledge, we can use Equation 15 to calculate the successor matrix directly as $M = (I - \gamma T)^{-1}$ (Russek et al., 2017).

T, M and $\mathbf{v}$ depend on the policy ρ used to explore the environment. To get a good estimate of the true value function, we need to have a policy that facilitates sufficient

exploration of the state space. A simple choice is a policy that selects an action uniformly at random from $\mathcal{A}$. For such a policy, we can obtain a default state transition matrix:

$$\tilde{T} = \frac{1}{n_a} \sum_{a \in \mathcal{A}} B_a$$

where B_a is the transition matrix for action a. This yields the following estimate of the successor matrix:

$$\tilde{M} = (I - \gamma \tilde{T})^{-1} \tag{20}$$

In a POMDP, the agent lacks direct access to the hidden states and instead relies on observations obtained through interactions with the environment. Computing the successor matrix directly is therefore not possible. However, as discussed earlier, the agent can use inference to compute belief states, which are posterior probability estimates of the hidden state. While these belief states b_t are distributions over the entire state space, computing the successor representation requires a single state estimate $\hat{s}_t$ to allow us to update the appropriate row of the successor matrix. To obtain $\hat{s}_t$, we can either sample from b_t or select the state with the highest posterior probability. The successor matrix can thus be computed as a function of state estimates derived from belief states.

One of the main drawbacks of computing the EFE for all possible sequences of actions π is that there are an exponential number of such sequences. However, following (Millidge & Buckley, 2023), we can emulate Equation 17 for computing the value function from the successor representation, and define the expected free energy function $\nu(s)$ as

follows:

$$\nu(s) = \sum_{s'} M(s, s')G(s') \tag{21}$$

where $G(s')$ is the one-step EFE evaluated at state s'. Note the similarity to Equation 17, with the free energy $G(s')$ replacing the reward function $r(s')$. For planning, the agent can use these ν values for the next states s' to generate an optimal sequence of actions.

In summary, the agent first computes the successor matrix, either directly using random exploration and temporal difference learning, or using the transition matrix. Next, given the matrices A, B, and C, at each time step, the agent computes the ν values for the states that can be reached from a particular state when different actions are executed. Finally the agent can determine the action to take from a given state at each time step by picking the action that leads to the state with the highest ν value. Chaining together these actions produces the optimum policy π^*.

3.3 Learning Hierarchical States and Actions using the Successor Representation

As discussed in the previous section, the successor representation is useful for computing new value functions on the fly, but as the state space size increases, learning a full successor matrix becomes difficult. We address this problem by learning state and action hierarchies. Hierarchical approaches, such as using higher-level abstract actions (also known as options, Sutton et al., 1999) have been widely studied in reinforcement learning. A hierarchy that involves abstract states and actions can significantly reduce the number of planning steps required, though potentially at the cost of not having a fully optimal plan with respect to the lowest-level actions. Animals and humans appear

to use hierachical strategies, such as using a sequence of pre-learned habitual behaviors to solve a complex task (Friston et al., 2018; Tomov et al., 2020).

We illustrate our approach to learning hierarchical states and actions using the classic problem of navigating in a "Gridworld." Figure 3a shows an example of a 9×9 grid defining a Markov Decision Process (MDP). The MDP is defined as follows:

$$\mathcal{M}_1 = \{S, A, T, R\} \tag{22}$$

where S is the set of states corresponding to discrete locations (black squares in Figure 3a), A is the set of 4 discrete actions (attempt to move left (L), right (R), up (U) and down (D) in the grid), T is the Markov transition function:

$$T : S \times A \rightarrow S \tag{23}$$

determining the next state given the current state and action, and $R : S \rightarrow \mathbb{R}$ is the reward function specifying the a scalar reward for each state. We assume for simplicity that both T and R are deterministic. Note that T is determined by the structure of a particular Gridworld, i.e., where the walls are, and executing an action that moves the agent against a wall results in the agent remaining in the same state/location (e.g., action U, L or R in the starting state in Figure 3a.

A common task for the agent in this Gridworld ("Task 1") is to navigate to a specially designated state called the goal state. The goal state could potentially change from one scenario to the next. In our experiments, the reward function is defined as follows for any state $s \in S$:

$$R(s) = \begin{cases} +100 & \text{if } s \text{ is the goal state} \\ -0.1 & \text{otherwise} \end{cases} \tag{24}$$

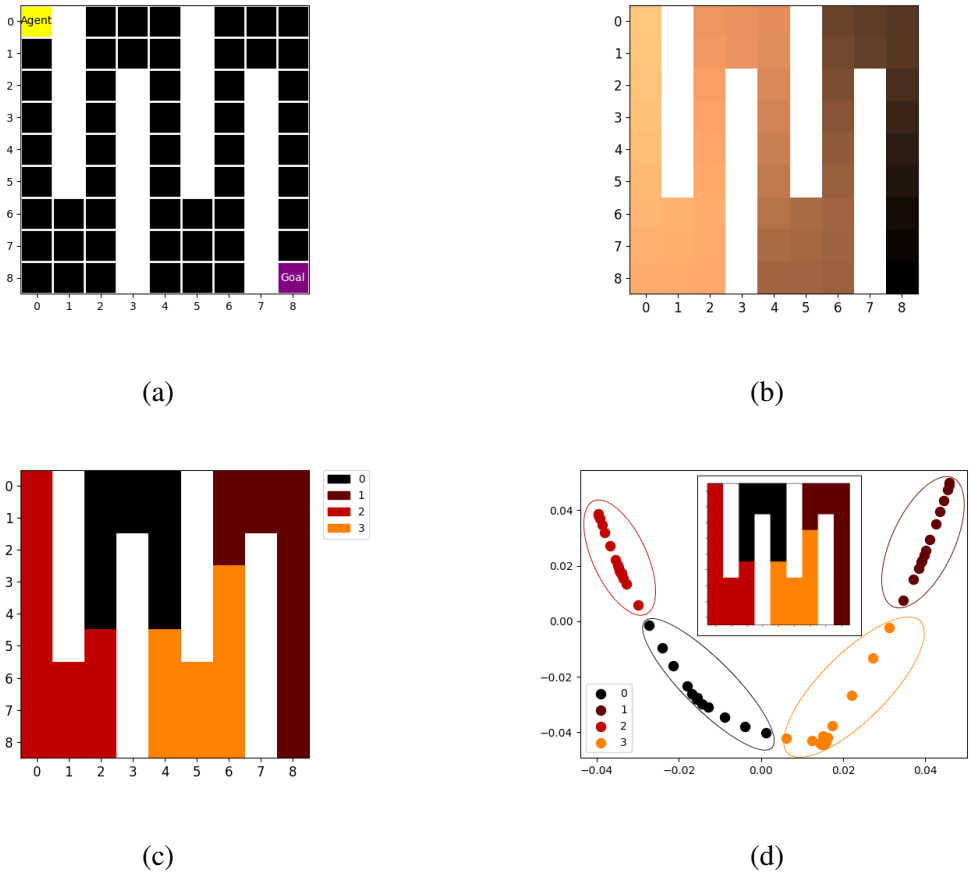


Figure 3: **Task 1: Serpentine MDP Gridworld Environment**

(a) A 9×9 grid world with a single start state (yellow), a single goal state (purple), and walls (solid white rectangles). (b) Successor representation of states with respect to the start state. The states closer to the start state have larger successor values and are represented by a lighter shade of brown. (c) Macro states learned by the model (4 in number). (d) Visualization of the 4 macro state clusters in terms of projections of the 81 micro states into an embedding space.

We use the negative reward (or "punishment") of -0.1 to encourage short paths to the goal.

Consider the problem of learning a 2-level state-action hierarchy for Task 1. We assume that the agent uses the following uniformly random action selection policy to explore the environment.

$$\text{For any } s \in S, \pi(s) = a \text{ with probability } \frac{1}{4} \text{ for the 4 possible } a \in A \tag{25}$$

The agent uses this exploration of the Gridworld and the resulting transitions (from a state s to state s' in each tine step) to learn a (potentially incomplete) successor matrix M using the temporal difference rule (Equation 19):

$$M(s,:) = M(s,:) + \alpha(\mathbf{1}_{s'} + \gamma M(s',:) - M(s,:)) \tag{26}$$

where $\mathbf{1}_{s'}$ is a one hot vector with value 1 at the position corresponding to state s'. Figure 3b shows the successor representation with respect to the start state s_0, i.e., the row $M(s_0,:)$, for the Gridworld in Figure 3a.

In order to learn abstract states at the higher ("macro") level, we cluster the lower-level ("micro") states using the successor representation as a *similarity metric between states*. We tested the following clustering methods:

- k-Medoids: This method is similar to k-Means, except the cluster centers are actual data points (Kaufman & Rousseeuw, 1990).

- Agglomerative Clustering: This is a bottom-up "hierarchical clustering" algorithm where each data point starts as its own cluster and the method iteratively merges the two most similar clusters until a specified condition is met (Murtagh & Contreras, 2012).

- Spectral Clustering: This method involves a spectral decomposition of the successor matrix M and uses the eigenvectors of a Laplacian matrix of M to reveal underlying clusters (Ng et al., 2002) (see also (Friston et al., 2025b) for an alternate approach to creating macro states based on renormalizing generative models).

In our experiments, we used spectral clustering because it yielded superior performance over the other two methods, in terms of requiring less training episodes (or equivalently less complete successor matrices) to get accurate clusters. To visualize the clusters in our figures, we used spectral embedding on the successor matrix (Pedregosa et al., 2011).

Figure 3c shows the four macro states $S_1, \ldots, S_4$ for Task 1 learned from spectral clustering of lower-level states (the number of macro states $k = 4$ was set a priori). Figure 3d shows a 2D visualization of the clusters of lower-level states.

Given the learned macro states (abstract states), the agent learns "macro actions" (abstract actions) for transitions from one macro state to another as follows. While exploring the environment, if the agent notices a transition from a micro state s_1 in macro state S_1 to a micro state s_2 that resides in a new macro state S_2, then s_2 is designated as a "bottleneck" state. The agent learns a macro action $A_{1\to 2}$ for the transition $S_1 \to S_2$ using active inference in the lower level to plan paths from any lower level state in S_1 to the bottleneck state s_2. The macro action $A_{1\to 2}$ is learned as a lower-level state-action policy $\rho_{1\to 2}$ using these paths computed using active inference. We repeat this learning process for pairs of adjacent macro states.

Given a set of macro states $S_1, \ldots, S_4$ and macro actions $A_{i\to j}$ for navigating from S_i to S_j, the agent can learn a successor matrix at the higher level, either by leverag-

ing past trajectories (used for learning macro states and actions) or by obtaining new trajectories directly from the environment, or both. Given these trajectories containing transitions from macro state S to macro state S', the higher level successor matrix is learned using the same temporal difference rule as Equation 19:

$$M(S,:) = M(S,:) + \alpha(\mathbf{1}_{S'} + \gamma M(S',:) - M(S,:)) \tag{27}$$

where $\mathbf{1}_{S'}$ is a one hot vector with value 1 at the position corresponding to state S'.

For active inference at the higher level, the agent requires EFE values for macro states. For each macro state S, the agent aggregates the one-step EFE values from all the micro states that the macro state encompasses:

$$G(S) = \sum_i G(s_i) \text{ for all } s_i \in S \tag{28}$$

In the case of a simple MDP, these EFE values are equivalent to the rewards of the states. This follows because there is no expected information gain (since the states are fully observable) and EFE reduces to expected reward (see Equation 5). Given the macro level successor matrix and the macro state EFE values, the agent can perform active inference with macro actions by first computing the EFE function $\nu(S)$ (Equation21) for all macro states S. Then, given any current state S_i, the agent selects the macro action $A^*_{i\to j}$ that will result in the macro state S_j with the highest ν value:

$$A^*_{i\to j} = A_{i\to j} \text{ such that } \nu(S_j) \geq \nu(S_k) \text{ for all } k \tag{29}$$

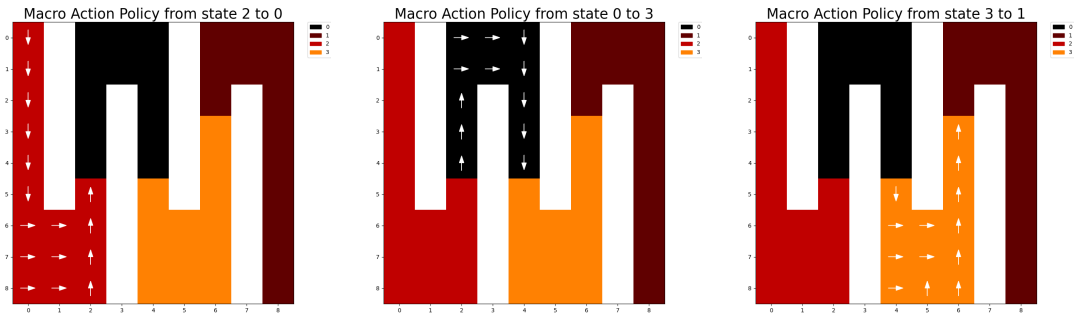
Finally, the agent executes the lower-level policy $\rho_{i\to j}$ corresponding to the macro action $A^*_{i\to j}$. This process is repeated until the agent reaches the goal state.

4 Results

4.1 Performance Comparison between Hierarchical and Non-Hierarchical Active Inference in Task 1

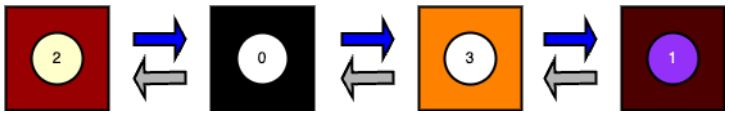
We compared the hierarchical active inference model described in the previous section with a model that uses active inference and the successor representation only at the lower-level (Millidge & Buckley, 2023) on Task 1. While the results presented here are for a particular Gridworld model for Task 1, we have obtained similar results for other wall configurations in Gridworld, and different locations of start and goal states (see Appendix 2.1 for results from a "Four Rooms" Gridworld environment).

Figure 4 shows three examples of macro actions learned by the model for the learned macro states in Figure 3. These macro actions are deterministic state-action policies that help navigate the agent from one macro state to an adjoining macro state. As described in the previous section, the agent learns these macro action policies through exploration and active inference. Each learned macro action guides the agent to a bottleneck micro state in the second macro state, from any micro state in the first macro state (for example, macro state 2 to macros state 0 in Figure 4a).

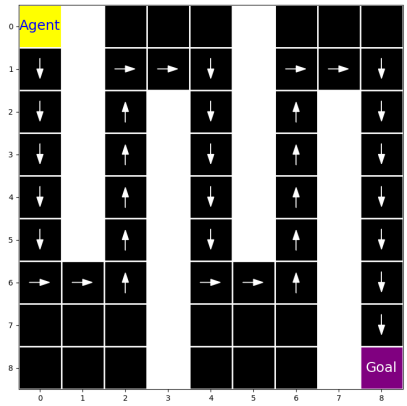


(a) (b) (c)

Figure 4: **Macro Actions for Task 1.** (a)-(c) represent macro actions that execute a policy at the lower level for transitioning from one macro state to another (here, macro actions for $S_2 \rightarrow S_0$, $S_0 \rightarrow S_3$, and $S_3 \rightarrow S_1$ are shown). The white arrows indicate the most probable action from each micro state in the first macro state for reaching the bottleneck state of the second macro state.



(a)



(b)

Figure 5: **Planning using Hierarchical Active Inference for Task 1.**

(a) Higher-level transition function and higher-level plan. The arrows represent the possible transitions between the learned macro states using the learned macro actions. The blue arrows represent the optimal sequence of macro actions (obtained using active inference at the macro level) to navigate from macro state 2 (which contains the agent's starting micro state) to macro state 1 (which contains the goal micro state).

(b) Lower-level action sequence executed by the agent as a result of higher-level plan. The arrows represent the micro actions executed by the agent based on the optimal macro action plan depicted by the blue arrows in (a), along with the micro actions used within macro state 1 to reach the goal. Compare the relatively large number of micro actions needed to reach the goal micro state to the much smaller number of macro actions (here, 3) that needed to be inferred with hierarchical active inference, illustrating the efficacy of planning using hierarchical states and actions.

Figure 5 illustrates the mechanics of hierarchical planning using the proposed model. The agent first learns the transition dynamics of higher level macro states and macro actions (Figure 5a) as part of the process of learning macro actions. This higher-level transition matrix can be used for higher-level planning using active inference: the blue arrows in Figure 5a show the optimal sequence of macro actions found by the model for navigating from macro state 2 (containing the start state) to the macro state containing the goal (macro state 1). The lower-level policy for each macro action in this sequence is then executed to navigate from one macro state to the next, ultimately reaching the last macro state in the sequence (macro state 1). Within macro state 1, the agent uses lower-level planning based on active inference to reach the goal micro state. Figure 5b depicts the sequence of micro actions resulting from this process, showing that the agent is able to successfully navigate from the start micro state to the goal micro state.

We compared the performance of our hierarchical model to a flat active inference model in learning a successor matrix and using the current learned matrix to navigate to the goal micro state. Also, as a baseline we used a simple Q-Learning model. Both types of models were tested for multiple random initializations. The average total reward obtained by each type of model is plotted in Figure 6a (the shaded regions correspond to +/- 1 standard deviation). Since the rewards are sparse, with only the goal giving a positive reward, the average total reward also behaves as a proxy for the average success rate of the model, so if the average reward is close to 100, that means that the agent always reached the goal, while if it is negative, the agent never reached the goal. As a sanity check we also evaluated a random policy that selects actions uniformly at random, without learning a value function. As shown in Table 1, the agent almost never

reaches the goal in the serpentine layout. Even with a generous budget of 1000 steps, the success rate is only 12.1%. Moreover, the average reward decreases with the step budget: although more steps provide additional opportunities to stumble upon the goal, the majority of episodes exhaust the full budget without success, accumulating a step cost of -0.1 per time step that far outweighs the occasional $+100$ goal reward.

Table 1: Random Policy: Success Rate and Average Reward vs. Maximum Steps Allowed

Max Steps Allowed	**Success Rate (%)**	**Avg Reward**
45	0.0	-4.50
100	0.0	-10.00
250	0.0	-25.00
500	1.5	-48.38
1000	12.1	-84.34

As seen in the figure, as the number of training episodes increases, the hierarchical model quickly leverages its current learned macro states and macro actions to achieve increasingly larger total reward, eventually reaching the goal state consistently after only about 750 training episodes (see Figure 6c). In contrast, the flat model fails to reach the goal even after several thousand training episodes due to the large number of planning steps needed at the micro action level to find a path to the goal location. The Q-learning fared somewhat better as compared to the flat model, but hierarchical active inference model was much better at successfully navigating to the goal.

How is the hierarchical model able to learn to navigate to the goal state so quickly compared to the flat model? The answer is suggested by Figures 6b and 6d, which plot the distance between the true successor matrix and the current learned successor matrix as a function of the number of training episodes. By comparing these plots to

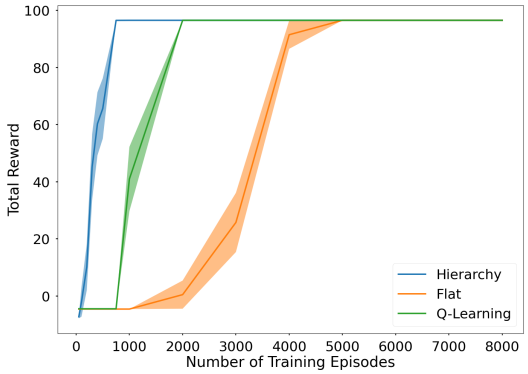
Figures 6a and 6c, we can see that the superior planning performance of the hierarchical model can be attributed to the much faster learning of the successor matrix at the macro level. Figure 6d illustrates how the macro level successor matrix can converge to the true matrix, even if the micro level successor matrix is not yet very accurate.

A final test of the two models for Task 1 involved moving the goal to progressively larger distances from the start micro state. As shown in Figure 6e, the hierarchical model scales much better, with modest increases in the number of planning steps as distance to the goal increases, whereas the flat model scales poorly and fails to reach the goal when distances are 25 steps or more.

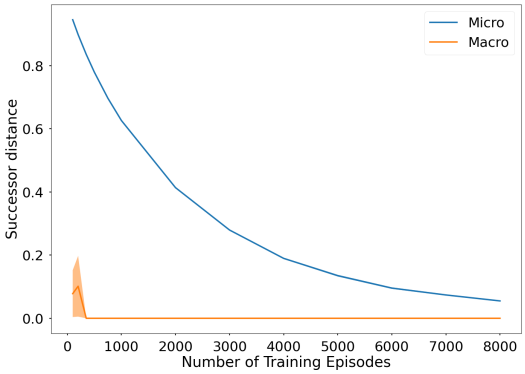
We additionally also test the learning stability of both the hierarchical and flat methods by a metric called R-stability introduced in (Ororbia & Mali, 2023). This metric penalizes post-learning performance degradation, by measuring the average normalized deviation of performance from the best achieved reward/return over the final phase of training. Lower values indicate that the agent after learning the policy maintains near-optimal behavior, i.e. it avoids any form of catastrophic forgetting. Higher values would indicate that model may be unstable or even collapse after transiently learning a good policy. As observed in Figure 6f, the hierarchical model demonstrates substantially lower relative stability than the flat baseline, indicating that once effective behavior is learned, it is consistently maintained. This suggests that hierarchical structure not only accelerates learning and planning, but also stabilizes behavior by reducing sensitivity to exploration noise, and local deviations in policy execution.

And finally, in order to show how good the agent is at replanning, we compare both Q learning and the SR methods in the case when there is a change in the goal, as seen

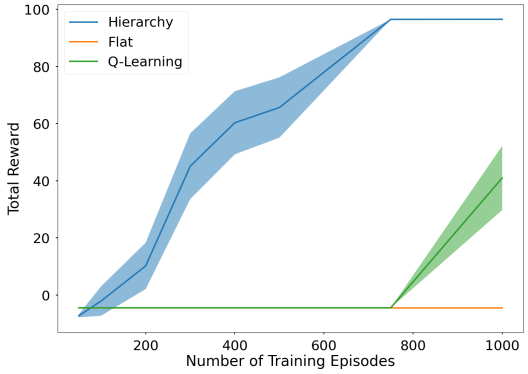
in Figure 7a. Clearly, the SR methods are instantly able to switch to the goal, while Q learning has to unlearn its existing values in order to be able to get to the new goal location, since it doesn't learn a successor matrix that is independent of the reward structure of the environment.



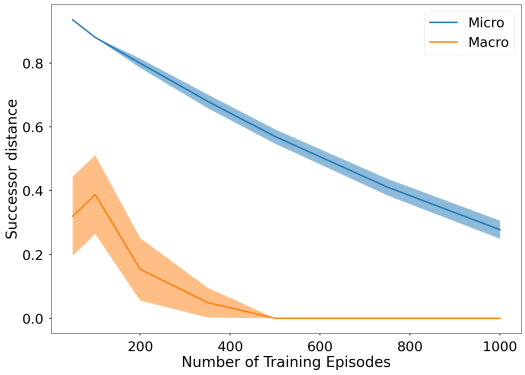
(a)



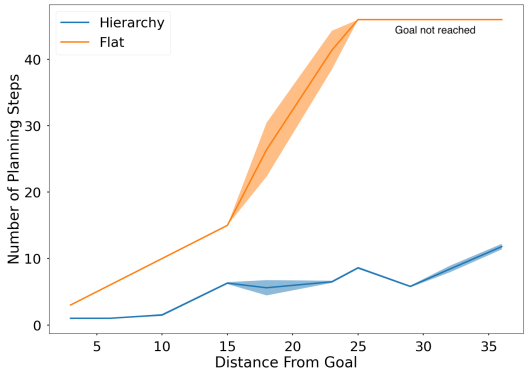
(b)



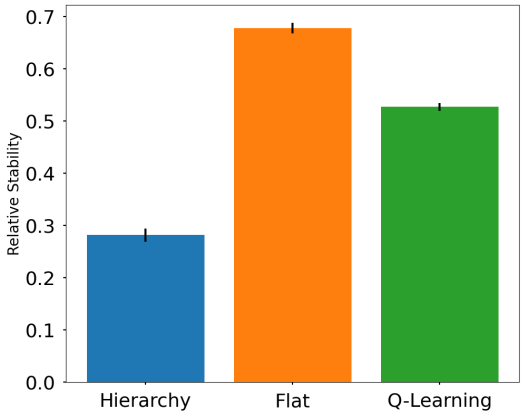
(c)



(d)



(e)

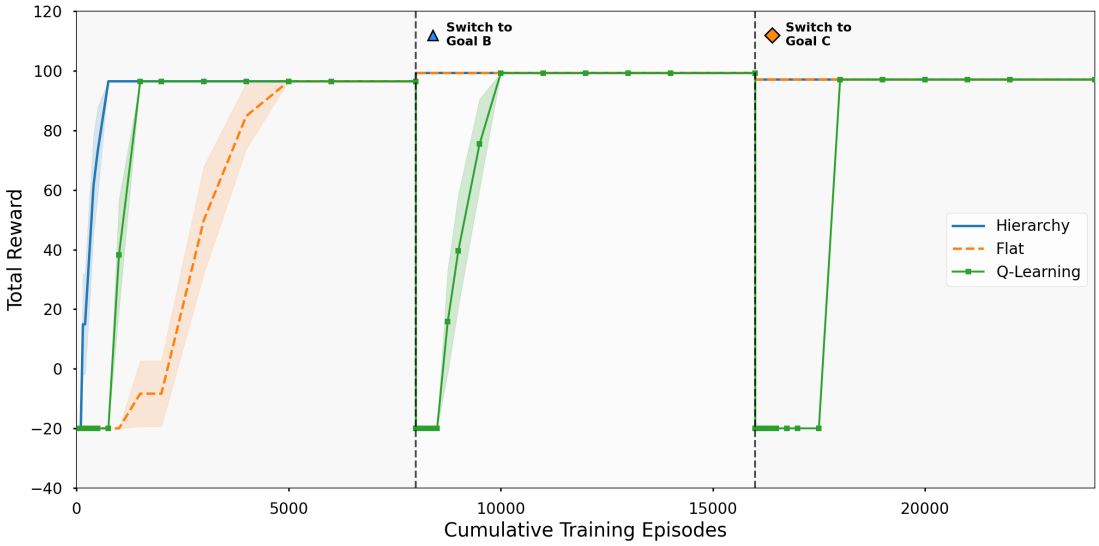


(f)

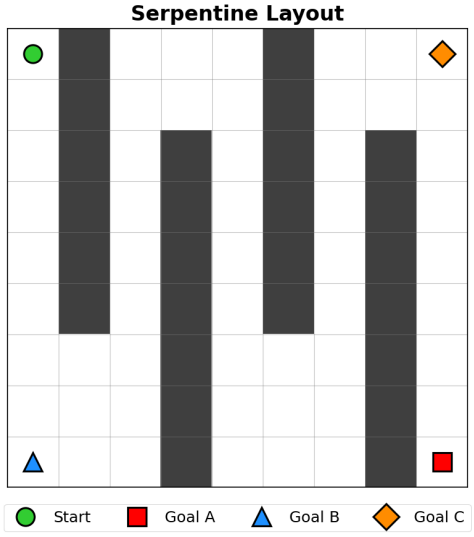
Figure 6: **Comparison of Planning Performance using Hierarchical, Flat Active Inference and RL baseline Models.**

(continued next page)

Figure 6: (a) Total reward obtained by the agent as a function of number of training episodes. The flat model takes $10\times$ the number of training episodes compared to the hierarchical model in order to reach the goal. The Q-Learning model performs better than the flat active inference but hierarchical model works much better. (b) Macro versus micro level successor matrix distances. The distance between the true successor matrix and the learned successor matrix at different stages of the learning process for a flat versus hierarchical agent. (c)-(d) Zoomed-in versions of (a)-(b). The plots show (a) and (b) for the first 1000 training episodes to highlight the initial behavior of each agent more clearly. (e) Number of planning steps needed to find the goal as a function of distance of the start micro state from the goal. For each distance, planning performance was tested given a fixed number of training episodes (1200). Note that the flat model fails to reach the goal for distances 25 steps or more from the goal (the agent was required to find the goal within a certain upper bound of planning steps). (f) The relative stability (RS) of both the hierarchical and flat methods (the lower the better) The hierarchical method is more stable ($RS = 0.281 \pm 0.013$) and maintains performance closer to its best achieved return than flat control ($RS = 0.677 \pm 0.010$), while the RL method is in between ($RS = 0.526 \pm 0.008$).



(a)



(b)

Figure 7: **Goal revaluation experiment.**

(a) Performance comparison between Q-learning and successor representation (SR) agents after the goal location is changed. The SR agent rapidly replans, whereas Q-learning requires substantial retraining. (b) Environment layout illustrating the sequence of goal relocations used during the experiment.

4.2 Task 2: Gridworld with Key

We next tested the hierarchical model on a Gridworld task with the added complexity of requiring the agent to have a key to be able to "unlock" the goal when it is reached and get the reward. Figure 8a illustrates this new task ("Task 2"): the agent must navigate to the location containing the key (which may be at a distance from the goal) and execute a new "pickup" action to get the key, before making its way to the goal. The action space for this task therefore consists of 5 actions: the four actions to move in each of the four directions and the pickup action.

Thus, if the number of locations in the Gridworld is n, then the number of micro states in the Gridworld with Key task is $2n$, assuming each micro state represents a location as well as whether or not the agent has the key. While a factored representation would be more efficient, for simplicity, we use here a one-hot vector with length $2n$ represent each state at the micro level.

Figures 8b-8d illustrate how the agent learns the macro states for Task 2. The learned successor representation (for the start state) is shown in Figure 8b: for visual depiction, the states corresponding to locations where the agent has not yet picked up the key are shown on the left, and the states corresponding to the same locations but after the key has been picked up are shown on the right. The results of clustering the successor representation to obtain macro states are shown in Figures 8c-8d (other macro states are also possible depending on choice of clustering method and parameters). Figure 9 shows four of the macro actions learned by the model to transition between the macro states in Task 2, based on the macro states learned in Figure 8. Note that given the increase in the size of the state space compared to Task 1, the agent requires more training episodes than in Task 1 for learning the macro states, macro actions, and successor matrix for active inference.

The macro level plan, obtained via active inference using the learned macro states and actions, is depicted by the blue arrows in Figure 10a (gray arrows represent other macro actions not used in this plan). The plan includes the macro action for transitioning from macro state S_0 to macro state S_4, which includes picking up the key (see Figure 9c). The execution of the macro level plan and the corresponding micro actions are depicted in Figures 10b and 10c, which show that the agent is able to successfully navigate to the key, pick it up, and reach the goal to unlock the reward.

4.3 Task 3: POMDP Gridworld

Our results thus far illustrate the efficacy of the successor representation in the MDP case. We now investigate hierarchical planning using active inference in the case where

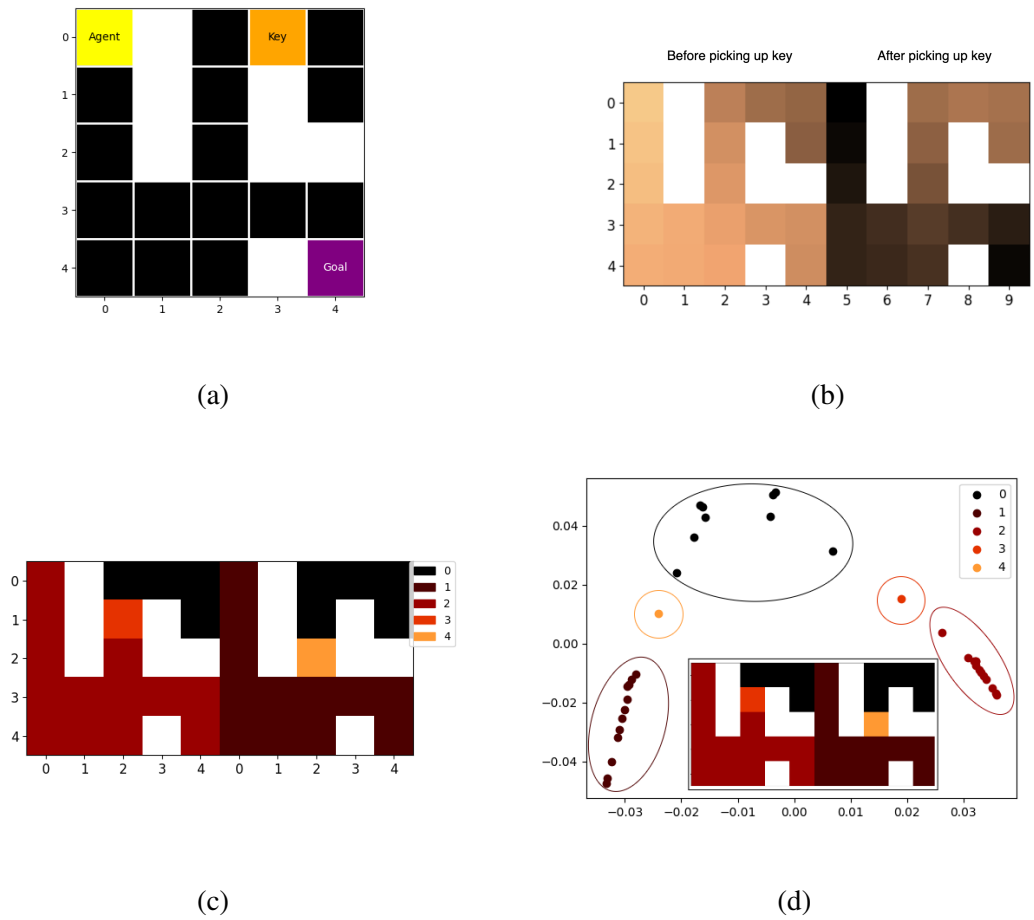


Figure 8: **Task 2: Gridworld with Key.**

(a) A 5×5 grid world with the agent's start location (yellow), goal (purple), key location (orange), and walls (white). (b) Successor representation with respect to start micro state (brighter is higher successor value). The left half corresponds to the micro states of the agent prior to picking up the key, while the right half corresponds to after. (c) Macro states learned by the model (number of macro states was set to 5). (d) Visualization of the 5 macro state clusters showing the micro states they contain.

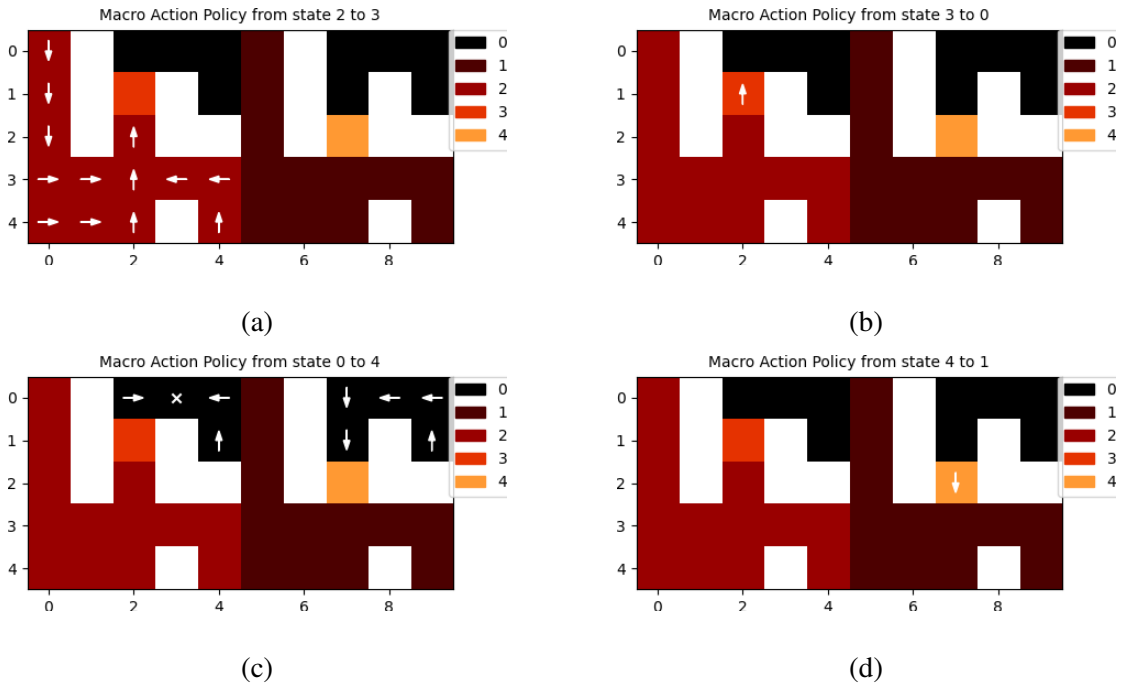
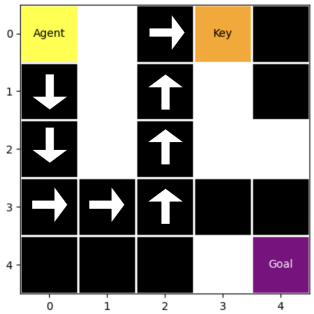


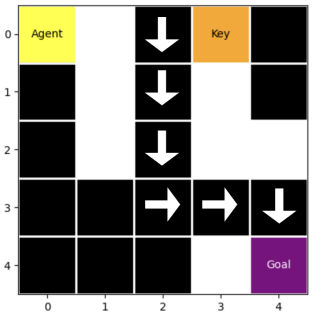
Figure 9: **Macro Actions for Task 2**.

(a)-(d) show four of the macro actions, in terms of their corresponding lower level policy, for transitioning from one macro state to another. The macro actions shown are for the transitions $S_2 \rightarrow S_3$, $S_3 \rightarrow S_0$, $S_0 \rightarrow S_4$ and $S_4 \rightarrow S_1$ (S_i represents macro state i in the plots). For each $S_i \rightarrow S_j$ macro action, the white arrows indicate the most probable action for each micro state within macro state S_i that enable the agent to reach the bottleneck state of S_j. 'x' represents the pick-up action.

(a)



(b)



(c)

Figure 10: **Planning using Hierarchical Active Inference for Task 2.**

(a) Higher Level Planning: The arrows represent macro actions between pairs of adjacent macro states. The blue arrows represent the macro action plan inferred by the agent through active inference at the macro level. This sequence of macro actions takes the agent from macro state 2 (containing the start location, no key) to 1 (containing the goal location, with key) through the macro state 0 which contains the location with the key. (b)-(c) show the execution of the plan in (a) in terms of the micro actions (white arrows) executed by the agent, allowing it to navigate from the start location to the goal, with a detour to pick up the key before going to the goal.

we have a partially observable environment. We modify Task 1 by introducing noise in the Gridworld environment.

In a standard MDP, the agent observes the hidden state exactly, meaning its observation space is identical to its state space. Now consider introducing some localized noise into the observation of the true state of the environment. Under this condition, for a given true state or grid location, the agent receives an observation that suggests it is located somewhere in the local neighborhood of the agent. To be more specific, in the likelihood matrix A that encodes $P(o|s)$, say o_k is the observation corresponding to the true state s_k and $o_{k'}$ corresponds to an observation in the local neighborhood of of s_k. We define the local neighborhood as any location one step to the left, right, up, or down from s_k (excluding walls). The noise is evenly distributed in this neighborhood. Then, given a noise parameter η, the likelihood is defined as $P(o_k|s_k) = 1 - \eta$ and $\sum_{k'} P(o_{k'}|s_k) = \eta$ where all the $P(o_{k'}|s_k)$ have equal values. The agent is assumed to have prior knowledge of this noise (e.g., through learning).

While active inference generally considers a partially observed environment (i.e., a POMDP), learning the successor matrix is not as straightforward as in the case of MDPs since the agent doesn't have access to the actual states. Instead the agent estimates a belief vector via state inference and uses that to update the successor matrix with the TD rule. Similar to the MDP case, the agent aggregates the EFE values of the micro states in order to compute the EFE for the macro states. In this case, the EFE values also depend on the entropy of the observational noise and not just the rewards of the state.

If we run our hierarchical model on a modification of Task 1 with added localized

observational noise, the agent is able to plan to navigate to the goal, with clusters similar to the MDP case (for results, see Appendix 2.2).

We focus on a new dimension introduced to the task when we convert the Gridworld MDP to a POMDP. A major advantage of using active inference for planning is that it naturally balances exploration versus exploitation. Here we investigate how our hierarchical agent balances these two aspects when navigating to a goal in our noisy POMDP environment.

Consider a Gridworld environment with five rooms, as illustrated in Figure 1 and shown again in Figure 11a for convenience. Assume there is some small general observational noise ($\eta = 0.1$) throughout the environment similar to what was described above. Additionally, we introduce comparatively larger noise distributed throughout the top right room in the environment (Figure 11a). Figures 11b-11d show the successor representation and clusters learned when we apply the hierarchical method. These clusters or macro states coincide with the 5 rooms in the environment when we set the number of clusters to 5. An interesting observation can be made in the embedding space of the micro states as seen in figure 11d. Even though the states corresponding to the corridors between pairs of rooms are assigned to one of the room clusters, they are located separately in the embedding space, indicating that they serve as a bottleneck state to which the agent can navigate to when executing a macro action. Figure 12 depicts some sample macro actions learned by the agent to navigate from one macro state to another.

If the amount of noise introduced in the top right room is sufficiently high compared to the rest of the environment, we expect active inference to select a path that avoids

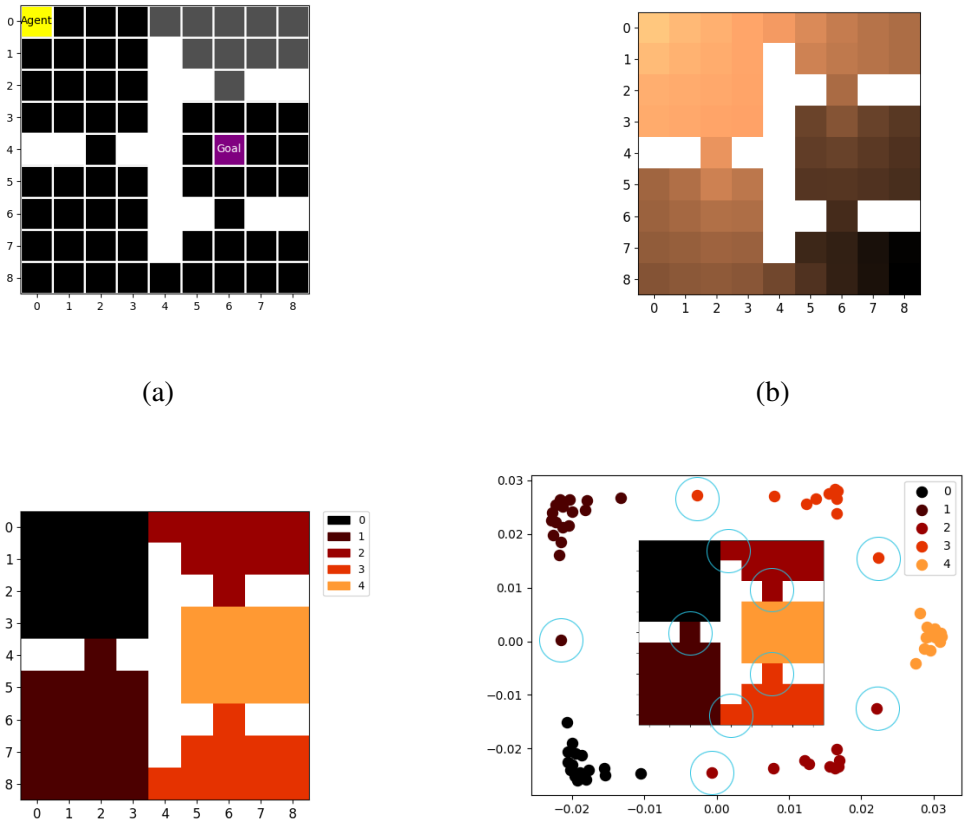
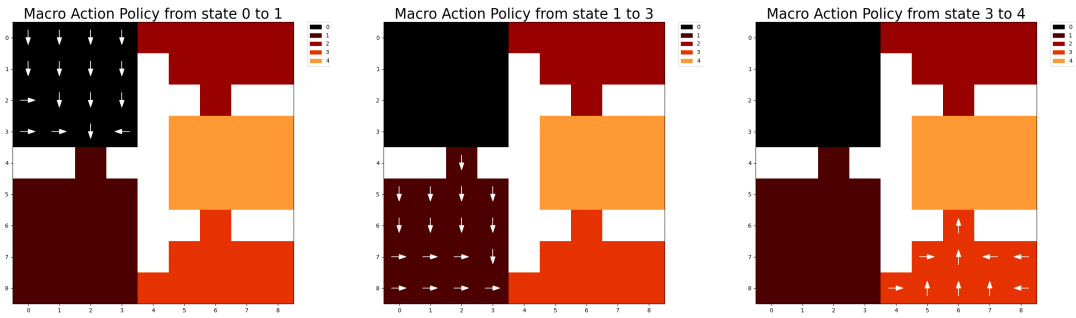


Figure 11: **Task 3: POMDP Gridworld Environment with five rooms.** (a) A 9×9 grid world with a single agent (yellow), a single goal (purple), walls (white) and high noise room (gray) with a reward of 100 associated with the goal and overall observational noise parameter $\eta = 0.1$. (b) Successor representation of states with respect to initial state (brighter is closer). (c) Macro states learned by the model, which coincide with the 5 rooms in the model. (d) Visualization of the 5 macro state clusters in terms of projections of the micro states into an embedding space. Note that the corridors between pairs of the 5 rooms have some separation compared to the rest of the clusters in the embedding space, as indicated by the blue circles.



(a) (b) (c)

Figure 12: **Macro Actions for Task 3.**

(a)-(c) represent macro actions that execute a policy at the lower level for transitioning from one macro state to another (here, macro actions for $S_0 \rightarrow S_1$, $S_1 \rightarrow S_3$, and $S_3 \rightarrow S_4$ are shown). The white arrows indicate the most probable action from each micro state in the first macro state in order to eventually reach the bottleneck state of the second macro state.

the noisy room due to the uncertainty associated with it, even if it means accruing some negative reward in traversing a longer path to reach the goal. Figures 13 and 14a demonstrate that this is indeed the case: the agent opts for the longer path to the goal because it offers a lower EFE value, optimizing the tradeoff between the entropy of the noise in the top right room versus the decrease in reward due to the longer path. When the noise in the top right room is decreased to the overall noise level across the environment, active inference prefers the shorter path (Figure 14b).

To explore the tradeoff optimized by active inference between avoiding high uncertainty versus maximizing rewards, we varied the level of noise in the top right room by progressively increasing the entropy of the observation model. Figure 15 shows that

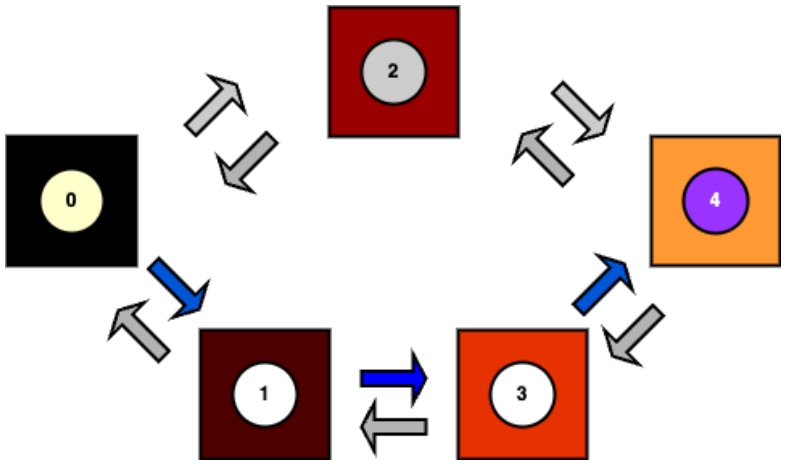
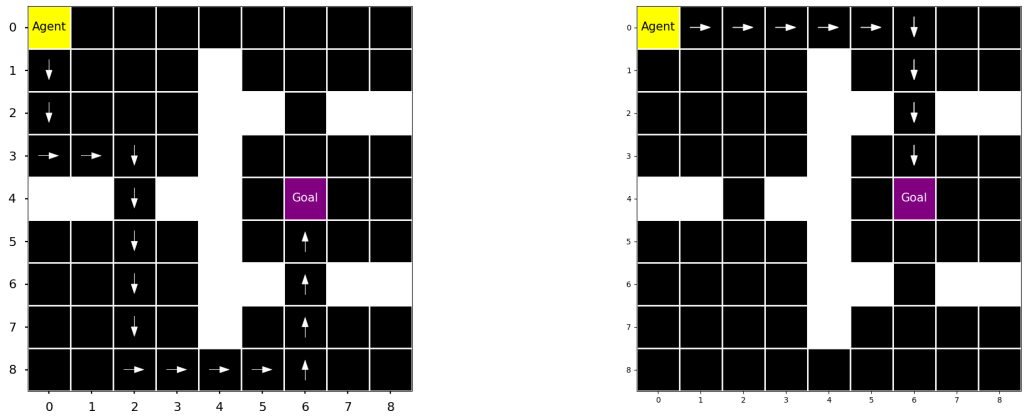


Figure 13: **Higher Level Planning for Task 3.** The arrows represent the potential macro actions between all pairs of adjacent macro states. The blue arrows represent the sequence of macro actions selected via higher-level active inference to get from macro state 0 (containing the start state) to macro state 4 (containing the goal state). Note that with active inference, the agent avoids the shorter path containing macro state 2 due to its larger observational noise (higher uncertainty) and opts for the longer path instead.

for lower levels of noise, the agent picks the shorter path to the goal but once the entropy crosses a particular threshold, the agent opts for the longer path to the goal. The threshold itself depends on many factors, such as the value of the rewards/penalty for an action at each step, the level of noise across the environment compared to the noise in the top right room, and the discount rate used to learn the successor matrix.

5 Task 4: Mountain Car

We now consider a more complex classic control task from the OpenAI Gym suite, a benchmark collection of environments widely used in reinforcement learning (Brock-



(a) (b)

Figure 14: **Active Inference optimizes the Tradeoff between Uncertainty and Reward in Task 3.** (a) Minimizing EFE using hierarchical active inference results in the agent naturally avoiding macro states with high noise (top right room), reaching the goal via the slightly longer path through two other rooms. (b) When the noise level in the top right room is decreased to the same noise level as the rest of the environment, the agent picks the shortest path to reach the goal.

man et al., 2016). As in the previously studied gridworld navigation setting, this environment requires long horizon planning under delayed or sparse reward signals. In contrast to gridworld however, the agent operates in a continuous state space, thus requiring some additional considerations before the model can be used. Note that there is no observational noise, hence this is an MDP formulation.

The task is straightforward. There is a cart that lies in the valley between two hills. The goal is to reach the top of the right hill, but the agent lacks the acceleration required in order to drive directly to the goal. Instead, the agent must exploit the environment's

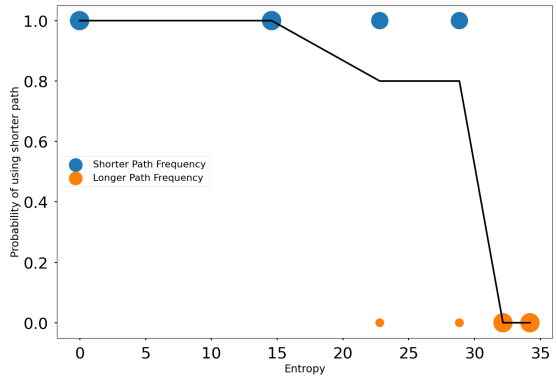


Figure 15: **Tradeoff between Uncertainty and Reward as a Function of Noise in Task 3.** The plot shows the probability that the agent, using active inference, chooses the shorter path to the goal in Task 3 as the noise in the top right room is increased by increasing the entropy of the observation model for that room. As the entropy increases, the agent increasingly prefers the longer path over the shorter one.

passive dynamics by first moving away from the goal to build up sufficient kinetic energy before ascending the right hill, as can be seen in Figure 16a.

The observation space is continuous and low dimensional, with position $x \in [-1.2, 0.6]$ and velocity $v \in [-0.07, 0.07]$. If the cart reaches $x = -1.2$ i.e. the leftmost point, then the collision is inelastic with the agent velocity set to 0. The action space is discrete and can be one of three options: Accelerate to the left, accelerate to the right, or no acceleration. The environment dynamics are deterministic, with each action applying a fixed acceleration magnitude and resulting in smooth but nonlinear state transitions due to gravity. An episode terminates when the cart position exceeds a threshold of $x \geq 0.5$ which corresponds to the top right hill, or when a maximum time horizon (200 steps) is reached. The reward signal is sparse, with the agent receiving a reward of -1 at each time step until termination, encouraging solutions that reach the goal in as few steps as

possible.

This environment is well suited for evaluating hierarchical and long-horizon planning methods, as the optimal policy requires non-greedy behavior and temporally extended action sequences.

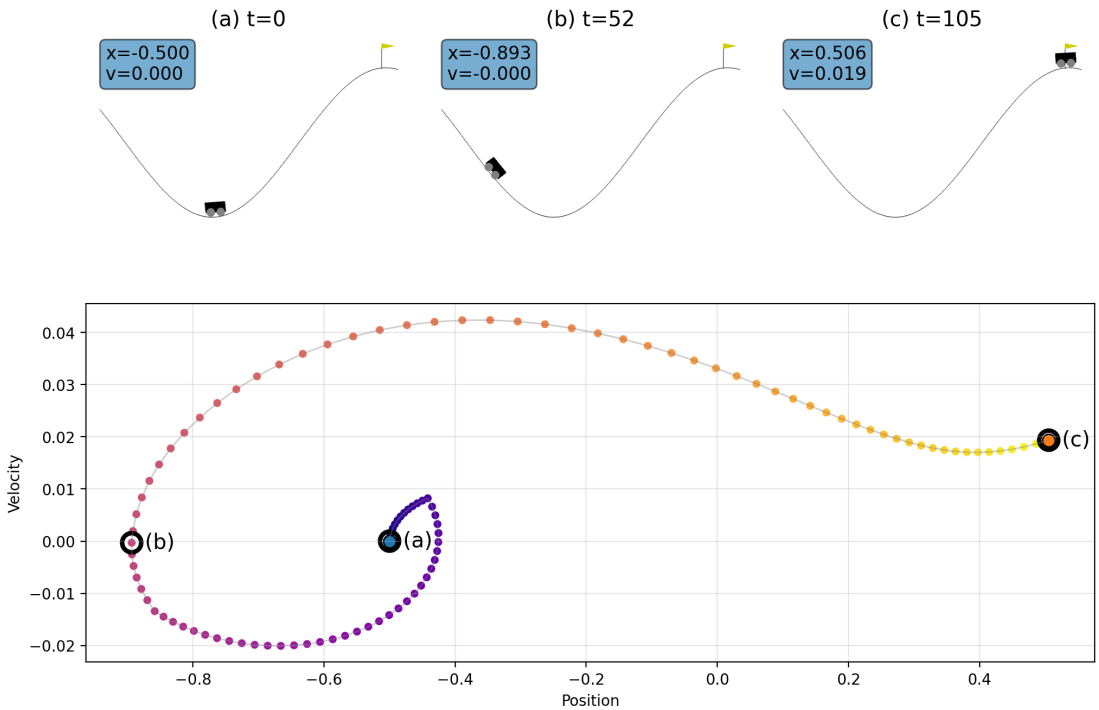
Although the mountain car environment is defined over a continuous state space, the successor based method that we have presented so far requires a discrete representation. This discretization enables the method to explicitly construct the hierarchical representations via clustering. Hence, we partition the continuous position and velocity dimensions into a fixed number of bins, yielding a discrete grid of micro-states that approximate the underlying dynamics. In the experiments, both the number of position and velocity bins are 10. As a result, the state space consists of 100 micro-states.

In our hierarchical active inference method, the successor representation serves a dual role: it is used to both construct a free energy based value function, as can be seen in Figure 16b, and also induce hierarchical structure through state clustering, as can be seen in Figure 17a. In the gridworld setting, we employed the full successor representation over the complete state space for both purposes without issue. In the case of this continuous problem, we found that there were often outliers formed in the clusters and they weren't as stable as in the gridworld case, and so an RBF kernel was applied as a regularizer. See Appendix 4 for more details.

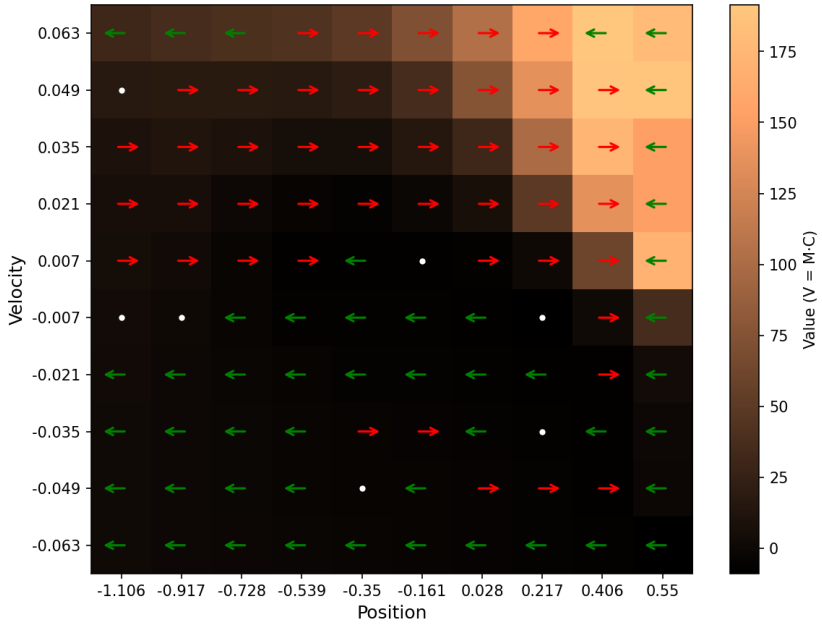
In order to make the model more robust, while randomly exploring the agent was directed to execute the same micro action k times in a row. (usually $k = 5$) This can be thought of as a fixed (priorly determined) macro-action as a level above the base micro-action. This allowed the agent to avoid additional instability while navigating

the environment and helped it explore the environment. The macro actions generated by the model via clustering were built on top of these intermediate fixed macro actions, and so the macro actions from one macro state to another can be thought of as sequences of the intermediate macro actions.

Clearly, as indicated by Figure 17d, during inference time when planning towards the goal, the agent only needs to plan on the higher level, and thus requires much fewer planning steps as compared to the flat model that must plan every single step using the free energy function.



(a)



(b)

Figure 16: **MountainCar environment and value function.**

(a) Key stages of the agent's trajectory in the MountainCar environment along with the associated state space. We show the agent at its start state, the leftmost turning point where it begins accelerating to the right, and the final goal location. (b) Learned value

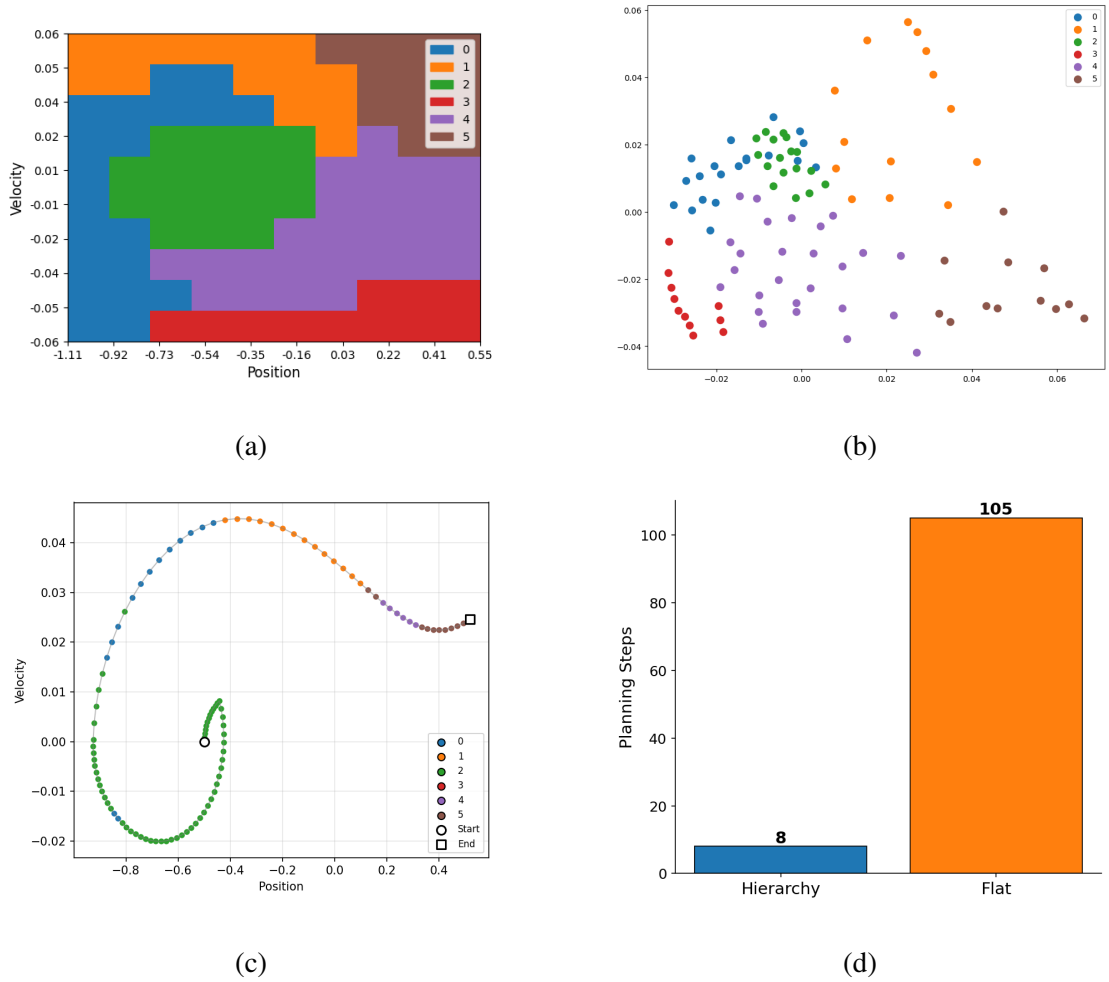


Figure 17: (a) **Clustering Heatmap** The 100 micro-states generated from discretization of the continuous state space were clustered into 6 macro-states using spectral embedding. (b) **Clustering Embeddings** Visualization of the macro states in the spectral embedding space (c) **Trajectory in the macro state space** Agent trajectory after assigning each micro-state to its macro-state cluster. (d) **Hierarchy vs Flat: Planning Steps** Hierarchical active inference requires substantially fewer action selections during inference by planning over temporally extended macro-actions instead of step-wise micro-action planning.

6 Task 4: Pendulum

The Pendulum environment (also based on OpenAI Gymnasium's `Pendulum-v1`) models a torque-limited inverted pendulum which is a classic control problem. The state space consists of the pendulum's angle $\theta \in [-\pi, \pi]$ and angular velocity $\omega \in [-8, 8]$ rad/s. The agent applies one of three discrete torques: negative (-2 Nm), zero, or positive ($+2$ Nm). The goal is to swing the pendulum from a hanging position ($\theta = \pi$) to the upright position ($\theta = 0$) and balance it there. Because the maximum torque is insufficient to directly lift the pendulum from rest, the agent must learn to build momentum through repeated swings, a strategy that emerges naturally from the learned dynamics rather than being explicitly programmed. Also, unlike previous problems, this is unique in that there is no end state as a goal, instead the agent must move and stay in certain portions of the state space based on a cost function that it receives at each time step. The reward structure mirrors Gymnasium's Pendulum cost function $r = -(\theta^2 + 0.1\,\omega^2 + 0.001\,\tau^2)$. Since the preference prior $\mathbf{C}$ is defined per state (torque τ varies per action), we use the state-dependent component

$$C(s) \;=\; -\big(\theta^2 + 0.1\,\omega^2\big), \tag{30}$$

rescaled to $[-\kappa,\, +\kappa]$ so that the upright equilibrium ($\theta{=}0,\ \omega{=}0$) receives the maximum preference $+\kappa$ and the worst configuration receives $-\kappa$. This positive–negative split is essential for the successor representation framework: the value $\mathbf{V} = \mathbf{M} \cdot \mathbf{C}$ must be positive at desirable states so that the agent is attracted toward them. An all-negative $\mathbf{C}$ would make $\mathbf{V}$ most positive at poorly connected edge states (with small $\mathbf{M}$ row sums), inverting the intended gradient.

The state space is discretized into a uniform grid of $\theta \times \omega$ bins. A key consideration is that the angle dimension is periodic: $\theta = -\pi$ and $\theta = +\pi$ represent the identical physical configuration (hanging straight down), yet they occupy opposite edges of the discretized grid. This periodicity must be respected during clustering to avoid artificially splitting dynamically connected boundary states into separate macro states. To address this, we embed each state in cylindrical coordinates $(\cos\theta, \sin\theta, \omega)$ before computing the spatial affinity kernel, ensuring that the distance metric wraps correctly at the angular boundary. See the Appendix 3 for the mathematical formulation. Also, similar to the mountain car problem, we combine it with a RBF spatial kernel for a regularizer.

The Pendulum's continuous dynamics, underactuation, and periodic topology make it a unique problem to solve. Unlike Mountain Car, where the state space has hard boundaries, the Pendulum's wrap-around angle dimension tests whether the clustering and macro-level planning can correctly identify macro states that span the angular boundary. The shaped reward landscape with a broad basin of attraction around the upright position also provides an opportunity to evaluate the agent's ability to plan hierarchically under dense, continuously varying preferences rather than sparse goal signals.

7 Conclusions and Future Work

Active inference has emerged as a promising biologically-plausible approach to embodied intelligence (Friston et al., 2017; Isomura & Friston, 2018) that balances aspects

of exploration and exploitation in a natural manner, and also brings action, learning, and perception together in a single unifying framework. However, implementations of active inference have been limited due to the difficulties in scaling traditional active inference methods to large-scale environments.

In this paper, we explored combining active inference with the successor representations to achieve a hierarchical and tractable framework for active inference that is more amenable to scaling. Specifically, the proposed approach learns both hierarchical states and hierarchical actions, enabling more efficient planning in large-scale environments through hierarchical active inference.

The hierarchical model was able to plan to the goal in navigation tasks using significantly less training as compared to a flat version of the model. Also, the model was able to learn a state hierarchy from the successor matrix even when the successor matrix was not complete enough for the agent to reach the goal using traditional low-level active inference.

We also presented results from a task that required picking up a key at one location and using it to unlock the reward at a different goal location. These results illustrate how state and action hierarchies can be learned by the model for tasks more complex than simple navigation to a goal location.

Using the POMDP five-rooms task, we showed how the agent naturally optimizes the tradeoff between uncertainty due to observational noise and expected reward by virtue of minimizing expected free energy. Hierarchical planning by active inference allowed the agent to choose between a shorter path to the goal and a longer path in the face of greater uncertainty in the observations. Hierarchical active inference was

previously applied to a different multi-rooms task (Neacsu et al., 2022), with a focus on concept formation and learning there is a limited number of rooms with a particular topology, rather than focusing on successor representations for efficient hierarchical planning as we do here.

While our preliminary results are encouraging, the hierarchical model is yet to be tested on more complex POMDP problems, and on continuous state and action spaces. For such scenarios, we intend to explore deep neural networks (Ueltzhöffer, 2018), focusing on their use for learning feature-based successor representations (Barreto et al., 2017) and learning hierarchical states and actions. We also intend to test the efficacy of hypernetworks in generating hierarchical state and action networks, paralleling their use in active predictive coding networks (Rao, 2024; Rao et al., 2024).

Although we only used a two-level model in this work, the model could be generalized to using three or more levels in the state-action hierarchy. Adding more levels could potentially make planning to achieve goals even more efficient and accurate, opening the door to applying the model to larger-scale POMDP problems.

Acknowledgments

This research was supported by the National Science Foundation (NSF) EFRI Grant no. 2223495, a Weill Neurohub Investigator grant, and a “Frameworks” grant from the Templeton World Charity Foundation.

Appendix

2 Gridworld

2.1 Four Rooms MDP Problem

In section 4.1, we examined a specific example of a Gridworld environment ("Serpentine") with a unique optimal path to the goal. Additionally, due to the placement of walls, this path is highly winding, requiring a large number of steps to traverse from the start state to the goal state. However, our method applies to any wall configuration as long as a valid path exists. Take for example the classic four rooms problem. Figure 18 illustrates how the agent plans in this scenario. This figure depicts the entire clustering and planning process, similar to Figure 3. Figure 18a depicts a 9×9 grid world containing a single agent (yellow), a single goal (purple), and walls (white). Figure 18b shows the successor representation of states, where states in closer proximity appear brighter due to their larger entries in the successor matrix. Figure 18c presents the macro states that are learned by the model by clustering based on the successor matrix. Since there are four rooms, we picked the same number of clusters; the agent learned a separate cluster for each room. Figure 18d visualizes these four macro state clusters by projecting the micro states into an embedding space. Note that in Figure 18d, the corridors connecting rooms have been separately marked by blue circles. While these corridors have been arbitrarily assigned to one of the four room clusters, one can observe how these states occupy a distinct region in the embedding space, effectively functioning as bottleneck states. Additionally, the agent's planning process at the macro level is shown

in Figure 18e while execution of the plan at the micro level is shown in Figure 18f.

2.2 POMDP Version of Task 1

We tested the model on a POMDP version of the Gridworld in Task 1, obtained by adding observational noise. The noise was localized, i.e., there is a small probability that the observation indicates that the agent is in one of the states in the local neighborhood of its true state. Figure 19a shows the 9×9 Gridworld used in the task, with a single agent (yellow), a single goal (purple), walls (white) and noise parameter $\eta = 0.2$. As explained in the main text, this η value means that 80% of the time the agent gets an observation that matches its true hidden state (true location), but 20% of the time the observation is a nearby state. Figure 19b shows the successor representation of states with respect to the initial state: states that are closer to the initial state show up brighter in the figure due to their larger entries in the successor matrix. Figure 19c shows the macro states that are learned by the model (4 in number). Figure 19d shows a visualization of the 4 macro state clusters in terms of projections of the micro states into an embedding space. These results show that for a low amount of noise, the method results in good clusters similar to the MDP case. If we significantly increase the noise to $\eta = 0.8$, the clustering process does get affected and the clusters are not contiguous anymore (Figures 19e and 19f). In this case, we found that the agent struggles to learn good macro actions, either getting stuck near a wall or entering into loops at the boundaries between the clusters.

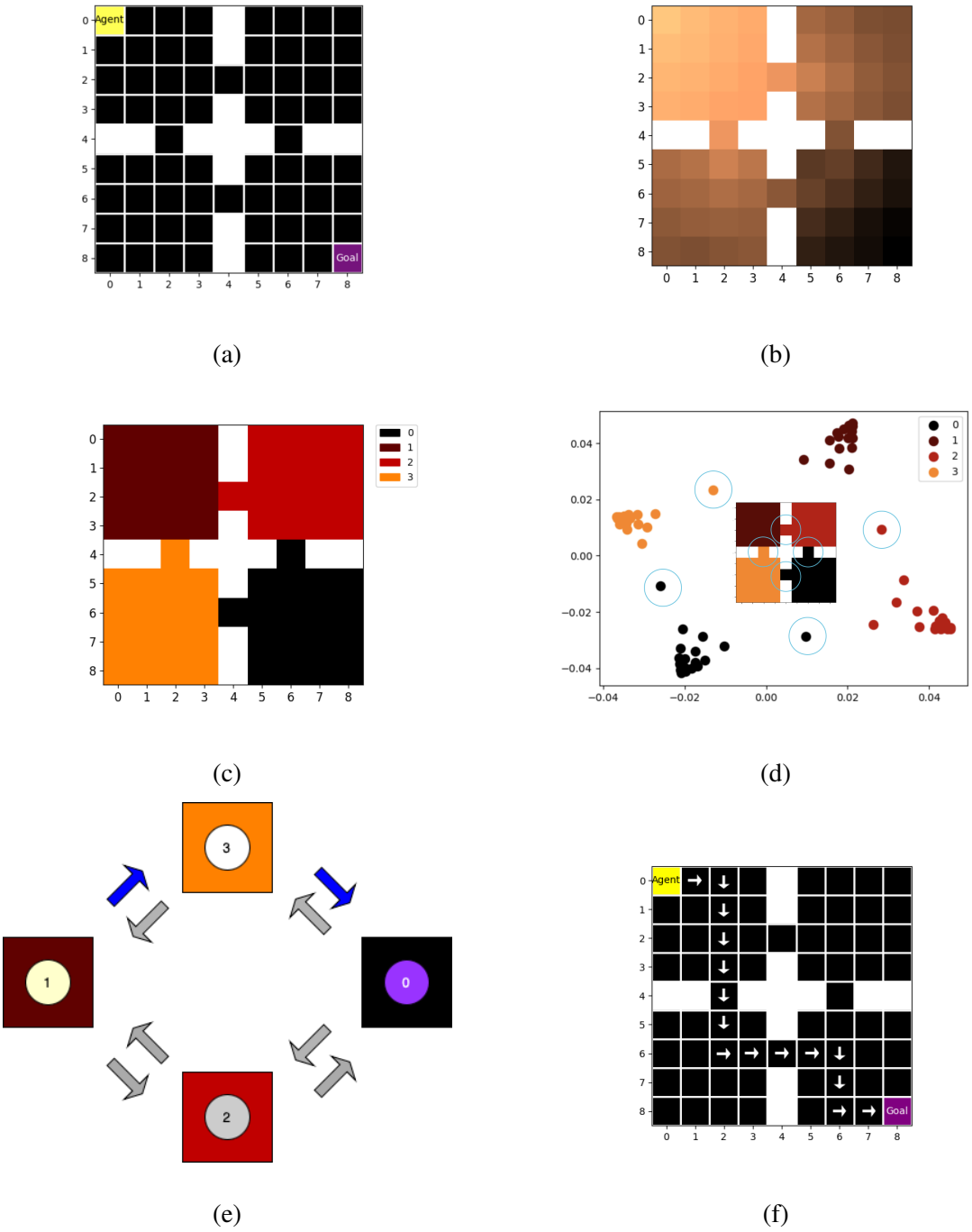
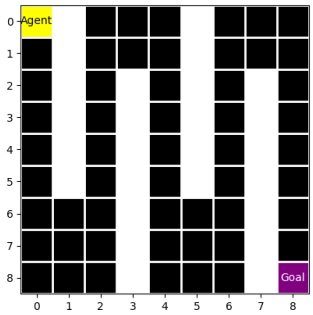
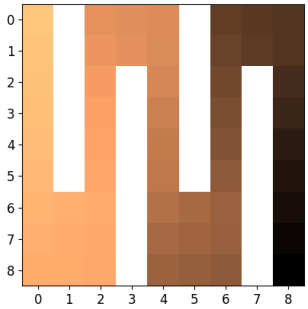


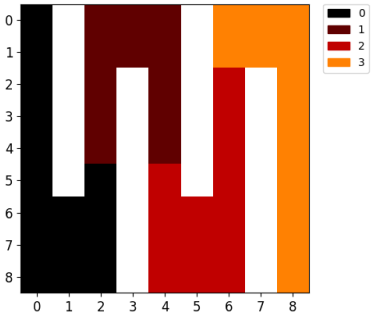
Figure 18: **Results for Four Rooms MDP Problem**. See text for details.



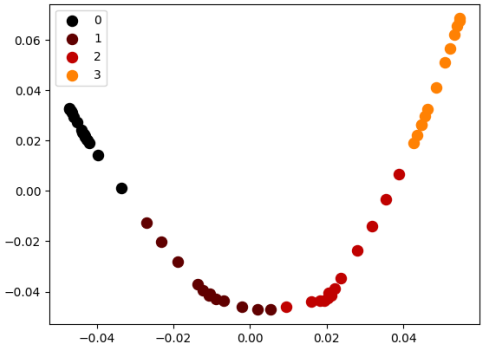
(a)



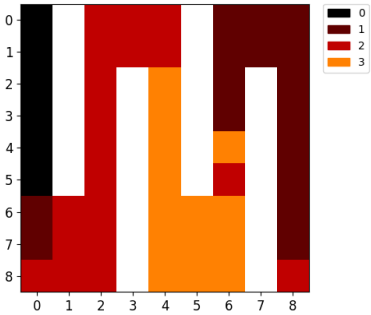
(b)



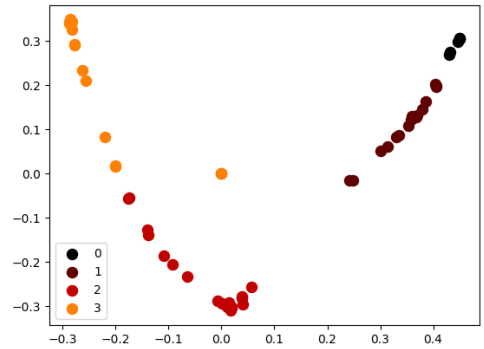
(c)



(d)



(e)



(f)

Figure 19: **Results for POMDP Version of Task 1**. See text for details.

3 Pendulum

The pendulum state space is parameterised by (θ, ω) where $\theta \in [-\pi, \pi]$ is *periodic*—the states $\theta = -\pi$ and $\theta = +\pi$ are physically identical (pendulum hanging straight down). However, in the discretised grid they occupy opposite edges, so standard spectral clustering on the successor representation matrix $\mathbf{M}$ can fragment dynamically-connected boundary states into separate clusters.

3.1 Cylindrical Embedding

Instead of operating in the flat (θ, ω) space, we embed each discrete state $s_{i,j}$ (with bin centres θ_i, ω_j) into a 3-dimensional *cylindrical coordinate space*:

$$\mathbf{x}_{i,j} = \left(\cos\theta_i,\ \sin\theta_i,\ \frac{\omega_j}{\omega_{\max}} \right) \tag{31}$$

where $\omega_{\max} = 8.0$ rad/s is the clipping bound for angular velocity. The angle maps onto the unit circle ($|\cos\theta|, |\sin\theta| \leq 1$), and dividing ω by $\omega_{\max}$ normalises it to $[-1, 1]$, making all three dimensions commensurate.

In this embedding, states at $\theta = -\pi$ and $\theta = +\pi$ map to the *same point* $(-1, 0, \omega_j/\omega_{\max})$, eliminating the boundary discontinuity.

4 Smooth Successor Matrices for Clustering

In the case of the experiments with continuous state spaces, often the exploration is not sufficient enough to get smooth clustering without outliers or discontinuities, which can affect the generation of macro actions and planning in the higher level. To address this

issue, we use a spatial RBF kernel as a form of regularizer on the matrix described as follows. In the case of the pendulum problem, the RBF kernel is applied on top of the cylindrical coordinates.

4.1 Spatial RBF Kernel

We compute pairwise squared Euclidean distances in the cylindrical space:

$$D^2_{(i,j),(k,l)} = \|\mathbf{x}_{i,j} - \mathbf{x}_{k,l}\|^2 \tag{32}$$

and form a radial basis function (Gaussian) kernel:

$$K^{\text{spatial}}_{(i,j),(k,l)} = \exp\left(-\frac{D^2_{(i,j),(k,l)}}{2\sigma^2}\right) \tag{33}$$

with bandwidth $\sigma = 1.0$. This kernel is symmetric, non-negative, and equals 1 on the diagonal.

For non-periodic state spaces (e.g. Mountain Car), the embedding reduces to normalised Cartesian coordinates and the kernel becomes a standard spatial RBF.

4.2 SR-Based Affinity

The standard spectral clustering affinity is the symmetrised successor matrix:

$$\mathbf{A}^{\text{SR}} = \max\big(\mathbf{M},\ \mathbf{M}^\top\big) \tag{34}$$

To blend it with the spatial kernel, we first normalise $\mathbf{A}^{\text{SR}}$ to $[0, 1]$:

$$\widetilde{\mathbf{A}}^{\text{SR}} = \frac{\mathbf{A}^{\text{SR}} - \min\big(\mathbf{A}^{\text{SR}}\big)}{\max(\mathbf{A}^{\text{SR}}) - \min(\mathbf{A}^{\text{SR}})} \tag{35}$$

4.3 Adaptive Blending

A uniform blend weight α applies the spatial kernel equally to all state pairs, regardless of how well $\mathbf{M}$ has been estimated. This can be problematic: in environments with walls or barriers, two states may be spatially adjacent but dynamically disconnected. A uniform spatial kernel would incorrectly boost their affinity, blurring the very bottleneck structure that clustering should discover.

We address this with an *adaptive* blend weight that varies per state pair based on the confidence in $\mathbf{M}$. The intuition is simple: trust $\mathbf{M}$ where it is reliable; fall back on the spatial kernel where it is not.

Per-state confidence. The row sum $r_i = \sum_j M_{ij}$ reflects how well state i has been explored: well-visited states accumulate large successor mass, while rarely-visited states (e.g. near-goal boundary regions) have small row sums. We normalise to $[0, 1]$:

$$\hat{c}_i = \frac{r_i}{\max_k r_k} \tag{36}$$

Per-pair confidence. For the pair (i, j), we take the minimum of the two states' confidences, so that the spatial kernel activates whenever *either* state is poorly estimated:

$$c_{ij} = \min(\hat{c}_i,\ \hat{c}_j) \tag{37}$$

Adaptive blend weight. The effective spatial weight for pair (i, j) is:

$$\alpha_{ij} = \alpha_{\max}\,(1 - c_{ij}) \tag{38}$$

where $\alpha_{\max}$ is the maximum blend weight (a hyperparameter; default 0.5 for the Pendulum, 0.3 for Mountain Car). The final blended affinity becomes:

$$\boxed{A_{ij} = (1 - \alpha_{ij})\, \widetilde{A}^{\text{SR}}_{ij} + \alpha_{ij}\, K^{\text{spatial}}_{ij}} \tag{39}$$

This gives the desired behaviour in three regimes:

- **Both states well-visited** ($\hat{c}_i, \hat{c}_j \approx 1$): $\alpha_{ij} \approx 0$. The affinity is determined almost entirely by the SR—dynamical barriers (walls, unreachable regions) are preserved.
- **At least one state poorly-visited** ($\min(\hat{c}_i, \hat{c}_j) \approx 0$): $\alpha_{ij} \approx \alpha_{\max}$. The spatial kernel fills in for the unreliable SR, preventing sparse boundary states from splitting off as outlier clusters.
- **Periodic boundaries (Pendulum)**: States at $\theta \approx \pm\pi$ are typically well-visited (the pendulum swings through the bottom frequently), so c_{ij} is moderate-to-high. The cylindrical kernel still provides a boost because the *spatial* distance across the branch cut is zero, while the SR may under-represent this connection due to bin discretisation. The adaptive weight ensures this boost is proportionate.

The key property is that $\mathbf{K}^{\text{spatial}}$ is invariant to the choice of angle branch cut—$\theta = -\pi$ and $\theta = +\pi$ have zero distance—so boundary states that are dynamically connected in $\mathbf{M}$ are no longer artificially separated by the discretisation.

4.4 Scope of Application

The blended affinity $\mathbf{A}$ is used exclusively for *macro-state discovery*: it determines the spectral embedding (for visualisation) and the spectral clustering (for cluster assign-

ments). The core micro-level quantities—the learned successor matrix $\mathbf{M}$, the transition matrix $\mathbf{B}$, and the value function $\mathbf{V} = \mathbf{M}\,\mathbf{C}$—remain unmodified.

Macro-level planning matrices ($\mathbf{B}_{\text{macro}}$, $\mathbf{M}_{\text{macro}}$) are derived from raw transition counts between the resulting clusters, so they are *indirectly* shaped by the blended clustering but do not incorporate the spatial kernel themselves.

This separation is deliberate: the spatial kernel should produce physically coherent macro-state regions without distorting the learned dynamics that drive action selection.

4.5 Related Work

Machado et al. (Machado et al., 2018) proved that the eigenvectors of the successor representation matrix $\mathbf{M}$ are equivalent to proto-value functions—the eigenvectors of the graph Laplacian over the state-transition graph. Their *eigenoption discovery* framework uses spectral decomposition of $\mathbf{M}$ to identify bottleneck states and construct temporally abstract actions (options). The earlier Laplacian framework (Machado et al., 2017) showed that eigenoptions correspond to eigenvectors associated with the largest eigenvalues of the graph Laplacian, providing a principled connection between spectral methods and option discovery in reinforcement learning. Our micro-level spectral clustering on $\mathbf{M}$ follows this established approach directly.

Clustering data embedded in spaces with periodic boundary conditions is a well-studied problem in computational science. The established solution is to embed periodic coordinates on the unit circle $(\cos\theta, \sin\theta)$ so that the metric respects the underlying topology. This approach has been applied to k-means (Gao et al., 2022) and DBSCAN (Lazzari et al., 2025) in molecular simulation and other periodic settings.

The idea of combining multiple affinity matrices appears extensively in multi-view clustering, where a unified affinity is learned from several "views" of the same data. Constrained spectral clustering modifies the affinity matrix with must-link / cannot-link constraints or spatial regularisation terms (von Luxburg, 2007). Our convex combination of a dynamics-based affinity ($\widetilde{\mathbf{A}}^{\mathrm{SR}}$) with a geometry-based kernel ($\mathbf{K}^{\mathrm{spatial}}$) is conceptually analogous, treating the successor representation and the physical state layout as two complementary views of the same state space.

To our knowledge, the specific combination of a learned successor representation with a cylindrical spatial RBF kernel via an adaptive, confidence-weighted convex blend for macro-state discovery has not appeared in prior work. The adaptive confidence weighting is a new contribution that makes the blending robust to environments containing dynamical barriers, where a uniform spatial kernel would incorrectly smooth over true bottleneck structure.

References

Aitchison, L., & Lengyel, M. (2017). With or without you: Predictive coding and Bayesian inference in the brain. *Current opinion in neurobiology*, *46*, 219–227.

Barreto, A., Dabney, W., Munos, R., Hunt, J. J., Schaul, T., van Hasselt, H. P., & Silver, D. (2017). Successor features for transfer in reinforcement learning. *Advances in neural information processing systems*, *30*.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). Openai gym. https://arxiv.org/abs/1606.01540

Chater, N., Zhu, J.-Q., Spicer, J., Sundh, J., León-Villagrá, P., & Sanborn, A. (2020). Probabilistic biases meet the Bayesian brain. *Current Directions in Psychological Science*, *29*(5), 506–512.

Da Costa, L., Parr, T., Sajid, N., Veselic, S., Neacsu, V., & Friston, K. (2020). Active inference on discrete state-spaces: A synthesis. *Journal of Mathematical Psychology*, *99*, 102447.

Dayan, P. (1993). Improving generalization for temporal difference learning: The successor representation. *Neural computation*, *5*(4), 613–624.

Dayan, P., & Hinton, G. E. (1992). Feudal reinforcement learning. *Proceedings of the 6th International Conference on Neural Information Processing Systems*, 271–278.

FitzGerald, T. H. B., Dolan, R. J., & Friston, K. (2015). Dopamine, reward learning, and active inference. *Frontiers in Computational Neuroscience*, *9*. https://doi.org/10.3389/fncom.2015.00136

Fountas et al., Z. (2020). Deep active inference agents using Monte-Carlo methods. *Advances in neural information processing systems*, *33*, 11662–11675.

Friston, K. J., et al. (2025a). Active inference and intentional behavior. *Neural Computation*, *37*(4), 666–700. https://doi.org/10.1162/neco_a_01738

Friston, K. (2010). The free-energy principle: A unified brain theory? *Nature reviews neuroscience*, *11*(2), 127–138.

Friston, K., Da Costa, L., Hafner, D., Hesp, C., & Parr, T. (2021). Sophisticated inference. *Neural Computation*, *33*(3), 713–763. https://doi.org/10.1162/neco_a_01351

Friston, K., FitzGerald, T., Rigoli, F., Schwartenbeck, P., & Pezzulo, G. (2017). Active inference: A process theory. *Neural computation*, *29*(1), 1–49.

Friston, K., Heins, C., Verbelen, T., Da Costa, L., Salvatori, T., Markovic, D., Tschantz, A., Koudahl, M., Buckley, C., & Parr, T. (2025b). From pixels to planning: Scale-free active inference. *Frontiers in Network Physiology*, *5*. https://doi.org/10.3389/fnetp.2025.1521963

Friston, K., Kilner, J., & Harrison, L. (2006). A free energy principle for the brain. *Journal of physiology-Paris*, *100*(1-3), 70–87.

Friston et al., K. (2015). Active inference and epistemic value. *Cognitive neuroscience*, *6*(4), 187–214.

Friston et al., K. (2018). Deep temporal models and active inference. *Neuroscience & Biobehavioral Reviews*, *90*, 486–501.

Gao, Z., et al. (2022). Using K-means clustering in Python with periodic boundary conditions. *Symmetry*, *14*(6), 1237. https://doi.org/10.3390/sym14061237

Gershman, S. J. (2018). The successor representation: Its computational logic and neural substrates. *Journal of Neuroscience*, *38*(33), 7193–7200.

Hafner, D., Lee, K.-H., Fischer, I., & Abbeel, P. (2022). Deep hierarchical planning from pixels. https://arxiv.org/abs/2206.04114

Huang, Y., & Rao, R. P. N. (2011). Predictive coding. *Wiley Interdisciplinary Reviews: Cognitive Science*, *2*(5), 580–593.

Isomura, T., & Friston, K. (2018). In vitro neural networks minimise variational free energy. *Scientific reports*, *8*(1), 16926.

Kaufman, L., & Rousseeuw, P. J. (1990). Partitioning Around Medoids (Program PAM). In *Finding Groups in Data* (pp. 68–125). John Wiley & Sons, Inc. https://doi.org/10.1002/9780470316801.ch2

Kingma, D. P., & Welling, M. (2022). Auto-encoding variational bayes. https://arxiv.org/abs/1312.6114

Knill, D. C., & Pouget, A. (2004). The Bayesian brain: The role of uncertainty in neural coding and computation. *TRENDS in Neurosciences*, *27*(12), 712–719.

Lazzari, F., et al. (2025). DBSCAN in domains with periodic boundary conditions. https://arxiv.org/abs/2501.16894

Li, L., Walsh, T. J., & Littman, M. L. (2006). Towards a unified theory of state abstraction for mdps. *AI&M*, *1*(2), 3.

Machado, M. C., Bellemare, M. G., & Bowling, M. (2017). A Laplacian framework for option discovery in reinforcement learning. *International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/1703.00956

Machado, M. C., Rosenbaum, C., Guo, X., Liu, M., Tesauro, G., & Campbell, M. (2018). Eigenoption discovery through the deep successor representation. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1710.11089

Millidge, B. (2020). Deep active inference as variational policy gradients. *Journal of Mathematical Psychology*, *96*, 102348.

Millidge, B., & Buckley, C. L. (2023). Active inference successor representations. *Active Inference: Third International Workshop, IWAI 2022, Grenoble, France, September 19, 2022, Revised Selected Papers*, 151–161.

Momennejad, I. (2020). Learning structures: Predictive representations, replay, and generalization. *Current Opinion in Behavioral Sciences*, *32*, 155–166.

Momennejad, I., Russek, E. M., Cheong, J. H., Botvinick, M. M., Daw, N. D., & Gershman, S. J. (2017). The successor representation in human reinforcement learning. *Nature human behaviour*, *1*(9), 680–692.

Murtagh, F., & Contreras, P. (2012). Algorithms for hierarchical clustering: An overview. *WIREs Data Mining and Knowledge Discovery*, *2*(1), 86–97. https://doi.org/https://doi.org/10.1002/widm.53

Neacsu, V., Mirza, M. B., Adams, R. A., & Friston, K. J. (2022). Structure learning enhances concept formation in synthetic active inference agents. *PLOS ONE*, *17*(11), 1–34. https://doi.org/10.1371/journal.pone.0277199

Ng, A. Y., Jordan, M. I., & Weiss, Y. (2002). On spectral clustering: Analysis and an algorithm. *Advances in neural information processing systems*, 849–856.

Ororbia, A., & Mali, A. (2023). Active predictive coding: Brain-inspired reinforcement learning for sparse reward robotic control problems. *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 3015–3021.

Pateria, S., Subagdja, B., Tan, A.-h., & Quek, C. (2021). Hierarchical reinforcement learning: A comprehensive survey. *ACM Comput. Surv.*, *54*(5). https://doi.org/10.1145/3453160

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Pezzulo, G., Rigoli, F., & Friston, K. J. (2018). Hierarchical active inference: A theory of motivated control. *Trends in cognitive sciences*, *22*(4), 294–306.

Proietti, R., Pezzulo, G., & Tessari, A. (2023). An active inference model of hierarchical action understanding, learning and imitation. *Physics of Life Reviews*, *46*, 92–118.

Rao, R. P. N. (2024). A sensory-motor theory of the neocortex. *Nature Neuroscience*, *27*, 1221–1235.

Rao, R. P. N., & Ballard, D. H. (1999). Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature neuroscience*, *2*(1), 79–87.

Rao, R. P. N., Gklezakos, D. C., & Sathish, V. (2024). Active predictive coding: A unifying neural model for active perception, compositional learning, and hierarchical planning. *Neural Computation*, *36(1)*, 1–32.

Rao, R. P. N., Olshausen, B. A., & Lewicki, M. S. (2002). *Probabilistic models of the brain: Perception and neural function*. MIT press.

Russek, E. M., Momennejad, I., Botvinick, M. M., Gershman, S. J., & Daw, N. D. (2017). Predictive representations can link model-based reinforcement learning to model-free mechanisms. *PLoS computational biology*, *13*(9), e1005768.

Sharot, T. (2011). The optimism bias. *Current biology*, *21*(23), R941–R945.

Smith, R., Friston, K. J., & Whyte, C. J. (2022). A step-by-step tutorial on active inference and its application to empirical data. *Journal of mathematical psychology*, *107*, 102632.

Stachenfeld, K. L., Botvinick, M. M., & Gershman, S. J. (2017). The hippocampus as a predictive map. *Nature neuroscience*, *20*(11), 1643–1653.

Stoewer, P., Schlieker, C., Schilling, A., Metzner, C., Maier, A., & Krauss, P. (2022). Neural network based successor representations to form cognitive maps of space and language. *Scientific Reports*, *12*(1), 11233.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.

Sutton, R. S., Precup, D., & Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, *112*(1-2), 181–211.

Tomov et al., M. S. (2020). Discovery of hierarchical representations for efficient planning. *PLoS computational biology*, *16*(4), e1007594.

Ueltzhöffer, K. (2018). Deep active inference. *Biological cybernetics*, *112*(6), 547–573.

Van de Maele, T., Dhoedt, B., Verbelen, T., & Pezzulo, G. (2023). Bridging cognitive maps: A hierarchical active inference model of spatial alternation tasks and the hippocampal-prefrontal circuit. *arXiv preprint arXiv:2308.11463*.

von Luxburg, U. (2007). A tutorial on spectral clustering. *Statistics and Computing*, *17*(4), 395–416.

Whittington, J. C., Muller, T. H., Mark, S., Chen, G., Barry, C., Burgess, N., & Behrens, T. E. (2020). The Tolman-Eichenbaum machine: Unifying space and relational memory through generalization in the hippocampal formation. *Cell*, *183*(5), 1249–1263.